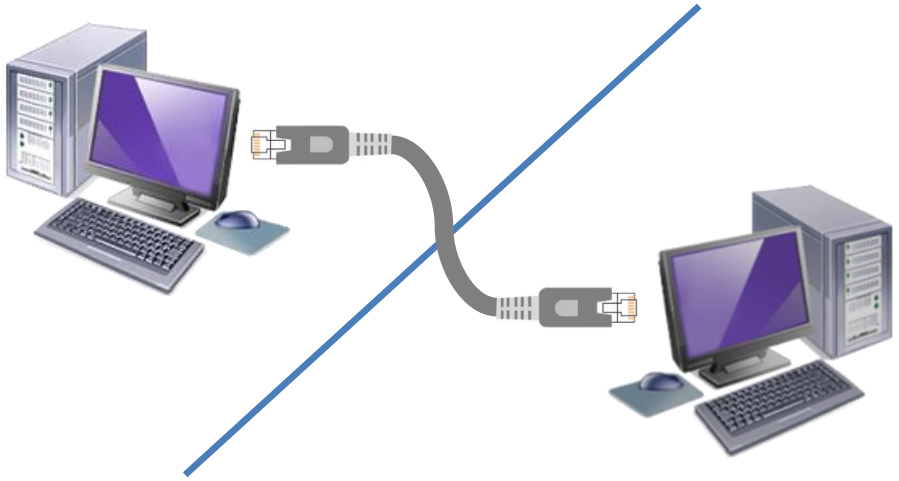
Communication réseau : Sockets



CNAM

Plan

- Rappels réseaux
- Rappels système
- Communication via sockets
- Présentation du projet

Rappels réseaux

Communication réseau

- Communiquer entre 2 machines nécessite
  - Référencer la machine de destination
  - Indiquer l'application réceptrice des données
  - Echanger des informations avec le même langage

Identification réseau

- Chaque machine a une identification unique sur un réseau (e.g., internet)
  - Adresse IP : série de 4 nombres entre 0 et 255
    - 163.215.82.55 (notation pointée)
  - Plusieurs adresses par machine (fonction destination)
    - Adresse interne (localhost ou boucle locale) : 127.0.0.1
    - Adresse sur le réseau local : 192.168.0.x
    - Adresse Internet (adresse du routeur vers extérieur) : 85.79.27.105
  - Adresses découpées en deux parties
    - Adresse du réseau et adresse de la machine (notation CIDR indiquant nb bits partie réseau)
- Obtenir son adresse
  - Locale : *ipconfig* (sous windows) / *ifconfig* (Linux/Mac OS)
  - Internet : par exemple *www.whatismyip.com*

Identification réseau

- Adresses classiques des machines utilisent IPv4
  - Adresse IP sur 32 bits : 4 nombres entre 0 et 255
    - 163.215.82.55 (notation pointée)
  - Milliards d'adresses possibles
- Problème : le nombre d'adresses avec IPv4 n'est plus suffisant !
  - Nécessité d'avoir une adresse par personne
- Migration de plus en plus vers IPv6
  - Adresse IP sur 128 bits : caractères en hexa, paire octets séparés par « : »
    - 2005:0db8:c9d2:aee5:73e3:924a:a5ae:9238
  - *Nb adresses supérieur au nb d'étoiles de l'univers*

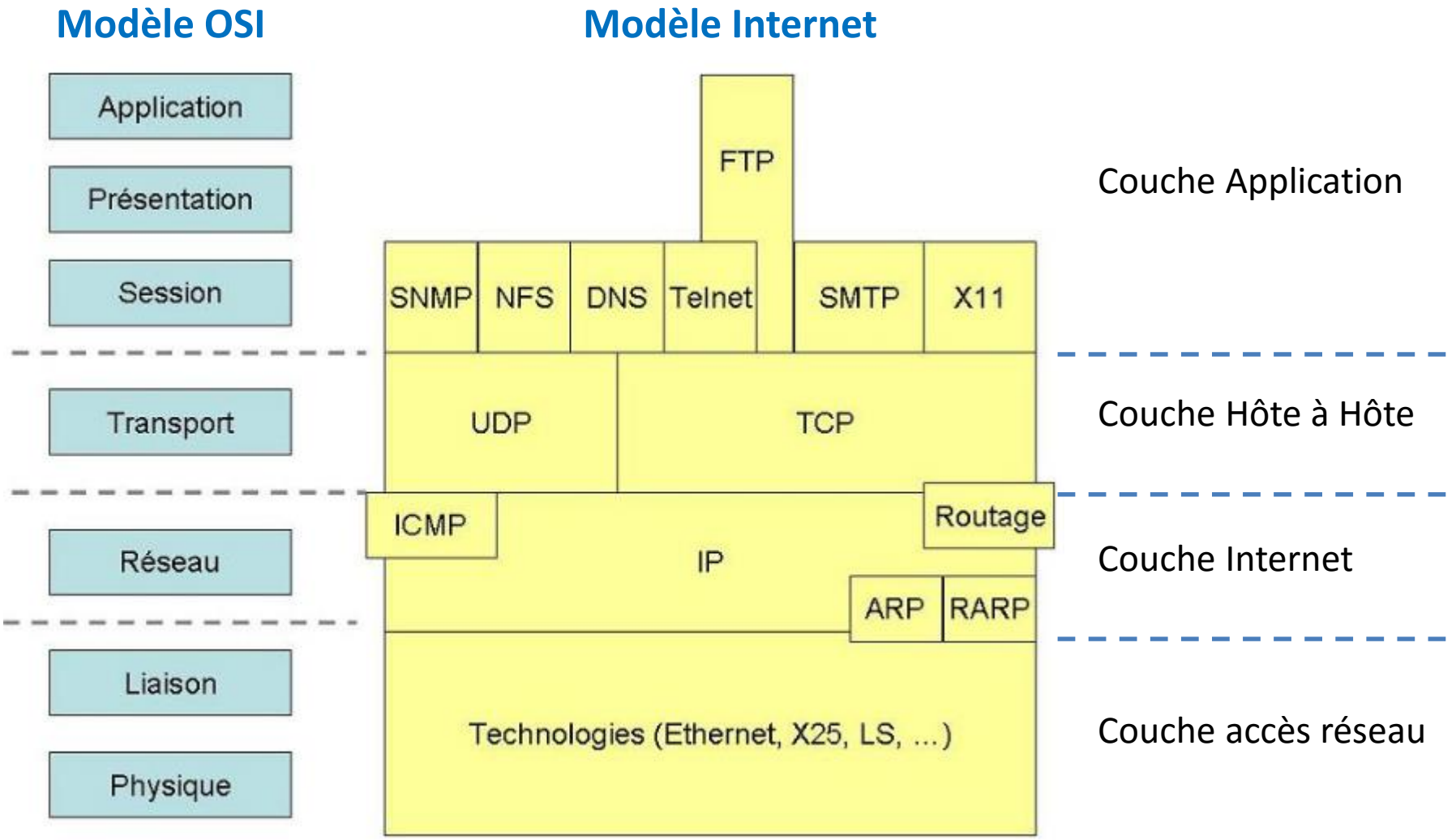
Ports réseau

- Plusieurs processus/services destinataires sur même machine
  - Données simultanément pour navigateur / client mail / jeux …
- Besoin d'indiquer le processus destinataire des informations
  - Utilisation **numéro de port** sur 16 bits (entier 1 à 65 536)
    - Ex : 21 pour FTP, 80 pour HTTP …
  - Protocoles/services fréquemment utilisés ont numéro de port dédié (indiqué dans */etc/services* sous Linux)
  - Utiliser un numéro de port libre (***supérieur à 1024***)
  - Rôle firewall : bloquer les ports non utilisés

Protocole réseau

- Protocole de communication
  - Ensemble de règles communes à 2 machines leur permettant d'échanger de l'information
- Certains nombre de protocoles existant
  - Haut niveau : manipulation spécifiée des données, documentation disponible (FTP, SMTP …)
  - Bas niveau : manipulation des données à définir (octet par octet), difficile mais grande liberté (TCP/IP, UDP/IP)
- **Tous protocoles haut niveau utilisent soit TCP/IP ou UDP/IP**

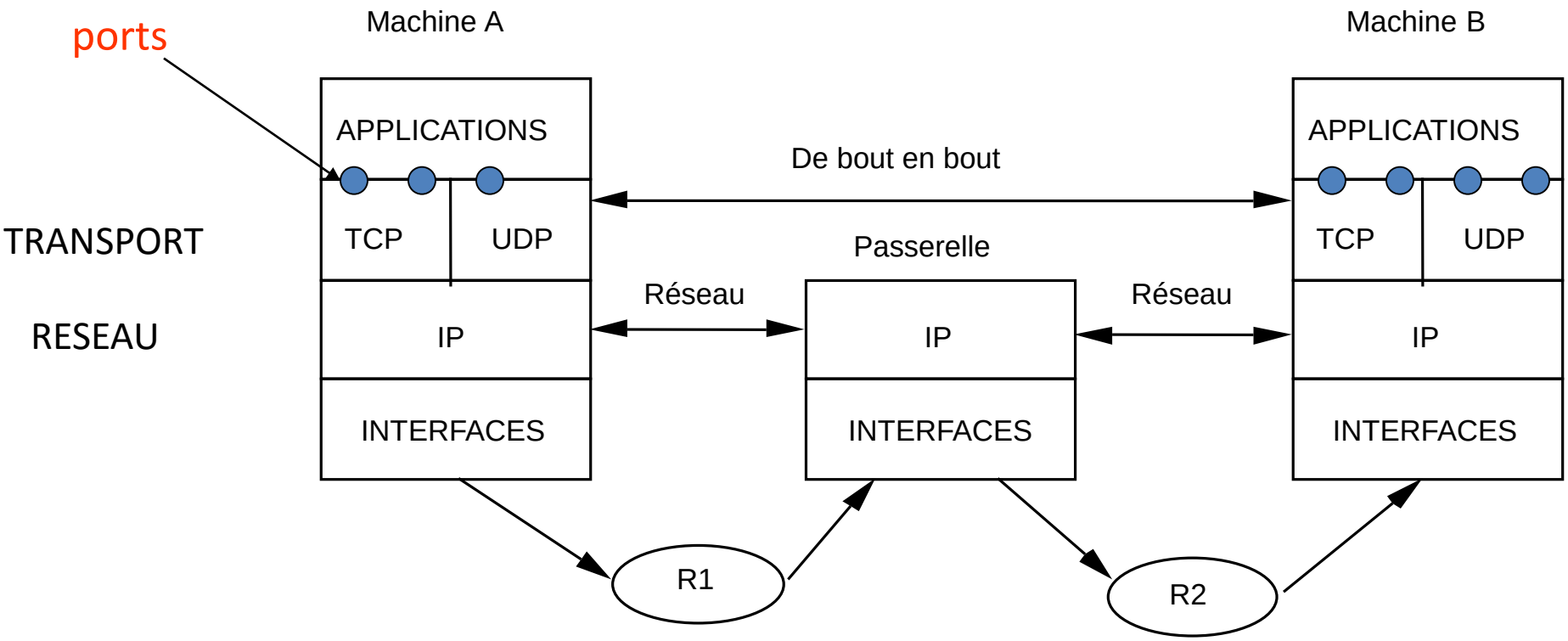
Couches réseaux



Quelques principes

- Echanges d'informations réalisés par morceaux
  - Niveau TCP/UDP : Envoie/réception de paquets de données
  - Niveau physique : paquets découpés en trames
  - Taille dépend de la couche MAC utilisée
- Deux méthodes d'échanges d'informations
  - TCP (Transmission Control Protocol) :
    - Etablie une connexion entre les 2 machines (**mode connecté**)
    - Système de contrôle des paquets, renvoie des paquets perdus
    - Echange plus coûteux et lent -> ajout d'informations de contrôle
  - UDP (User Datagram Protocol) :
    - Echange de données sans connexion (**mode non-connecté**)
    - Aucun contrôle des paquets (possibilité de perte ou désordonné)
    - Transmission des informations plus rapide (peu d'infos de contrôle)

Quelques principes

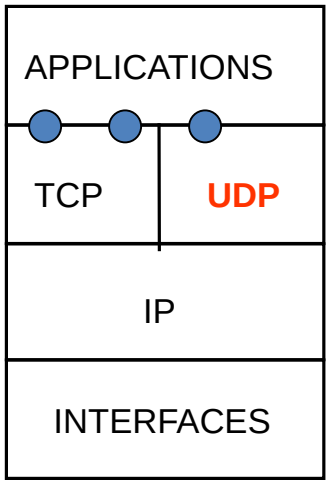


- IP : adressage de machine à machine via adresse IP
- TCP/UDP : adressage d'applications à applications
  - ➢ notion de ports (entier 16 bits)

La couche UDP (User Datagram Protocol)

- Protocole transport de bout en bout

  adressage d'application à application via les ports UDP
  Ports UDP réservés
  exemple : port 513 pour application who



- Protocole transport non fiable basé sur IP

  Datagrammes indépendants
  Perte de datagrammes, datagrammes dupliqués, pas de remise dans l'ordre d'émission

Analogie : le courrier

La couche TCP (Transport Control Protocol)

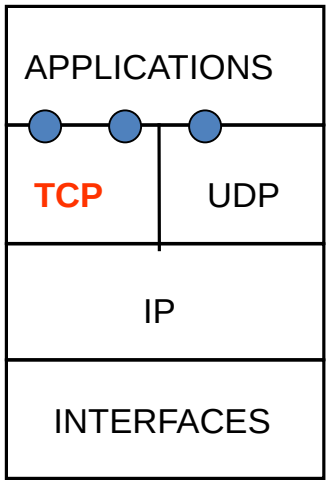
- Protocole transport de bout en bout

  adressage d'application à application via les ports TCP
  Ports TCP réservés
  exemples : port 21 application ftp

- Protocole transport fiable orienté connexion basé sur IP

  Connexion / Flux d'octets
  Pas de perte de messages, pas de duplication, remise dans l'ordre d'émission

  Analogie : le téléphone

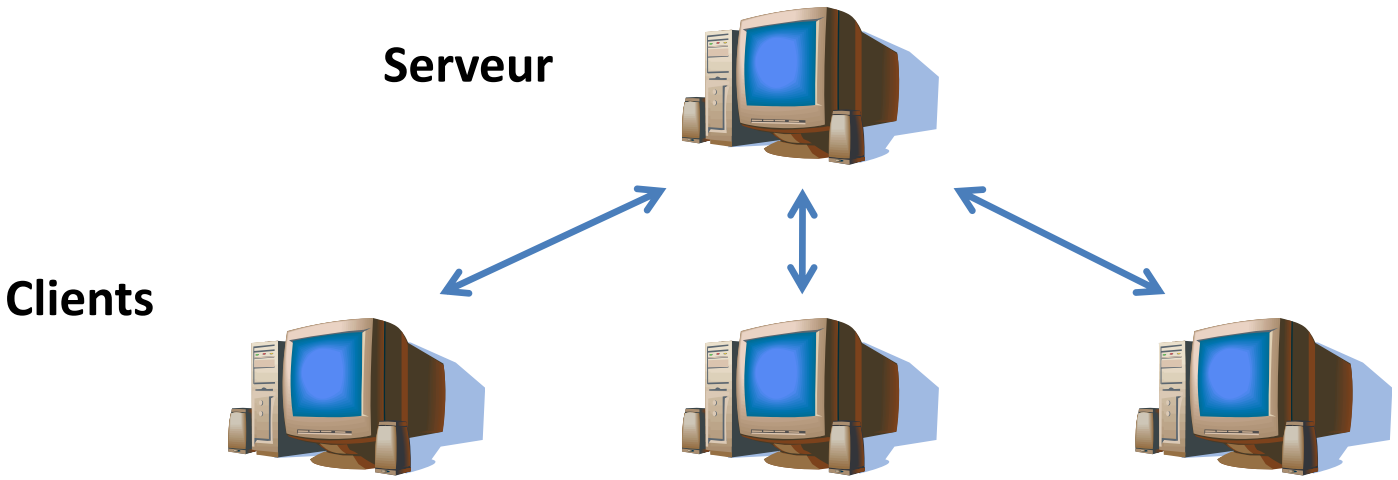


Utilisation TCP-UDP

- Quand utiliser TCP ?
  - Si perte d'informations gênante pour application
  - Contrôle des paquets
  - Ex : transfert de fichiers, de texte, …
- Quand utiliser UDP ?
  - Si perte d'informations non gênante et besoin de communications rapides
  - Aucun contrôle des paquets
    - Sinon utiliser TCP ou à la charge de l'application
  - Ex : jeux en ligne, contenu en streaming …

Architectures réseaux

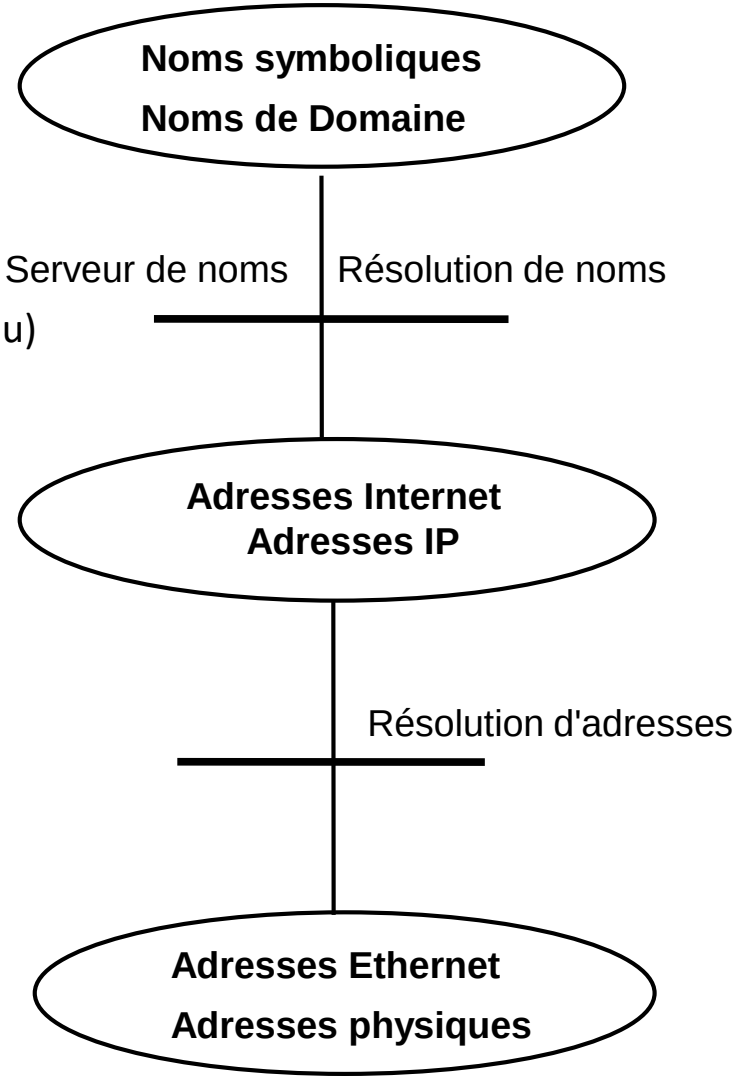
- Architecture Client/Serveur
  - Architecture la plus classique
  - Programmes clients communiquent via un programme serveur (au centre de l'architecture)



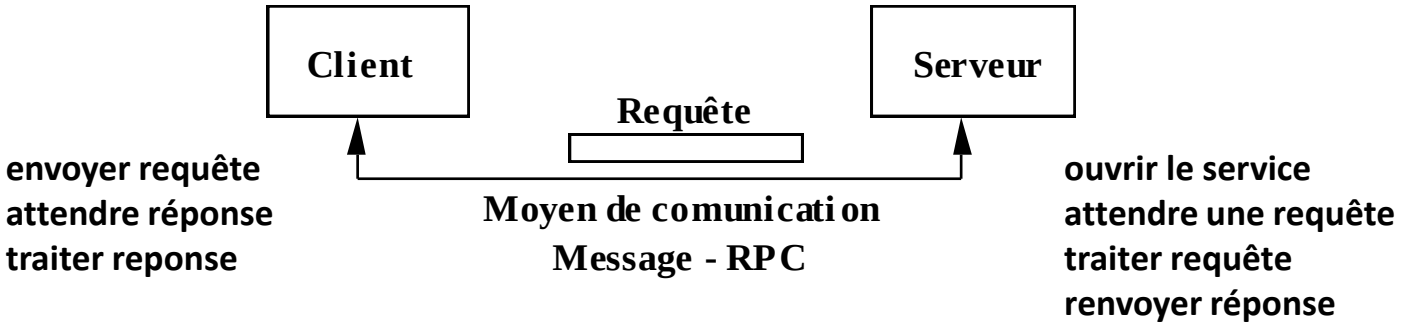
Architectures réseaux

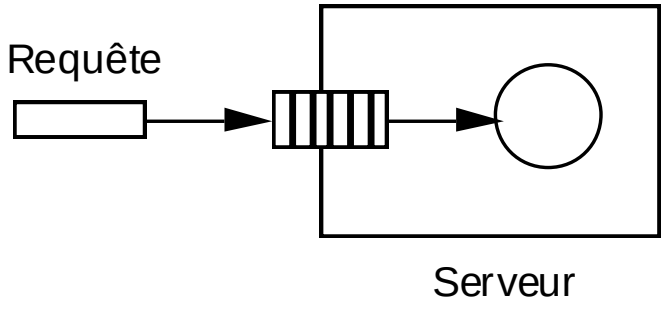
Structure hiérarchique fondée sur le domaine (DNS)
objet.sous-domaine.domaine fermi.cnam.fr
objet : nom d'une machine.
domaine : géographique (fr, jp) ou institutionnel (com, mil, edu)

Adresses IP : 32 bits
Paire (adresse réseau, adresse machine dans le réseau)
Forme pointée : ***10000000 00001010 00000010 00011110***
128.10.2.30

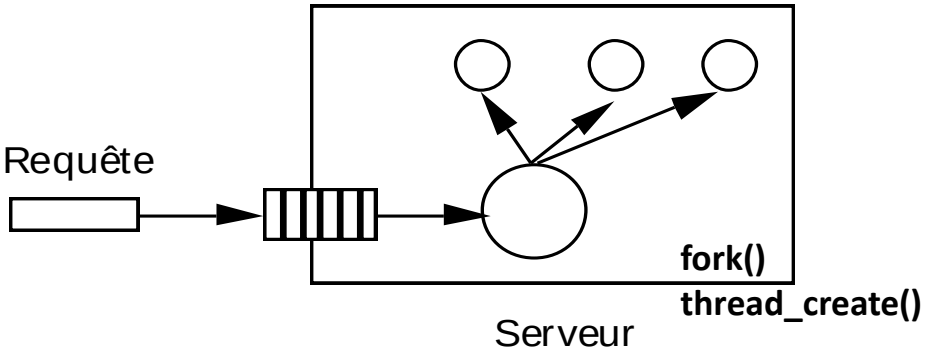


Architectures réseaux





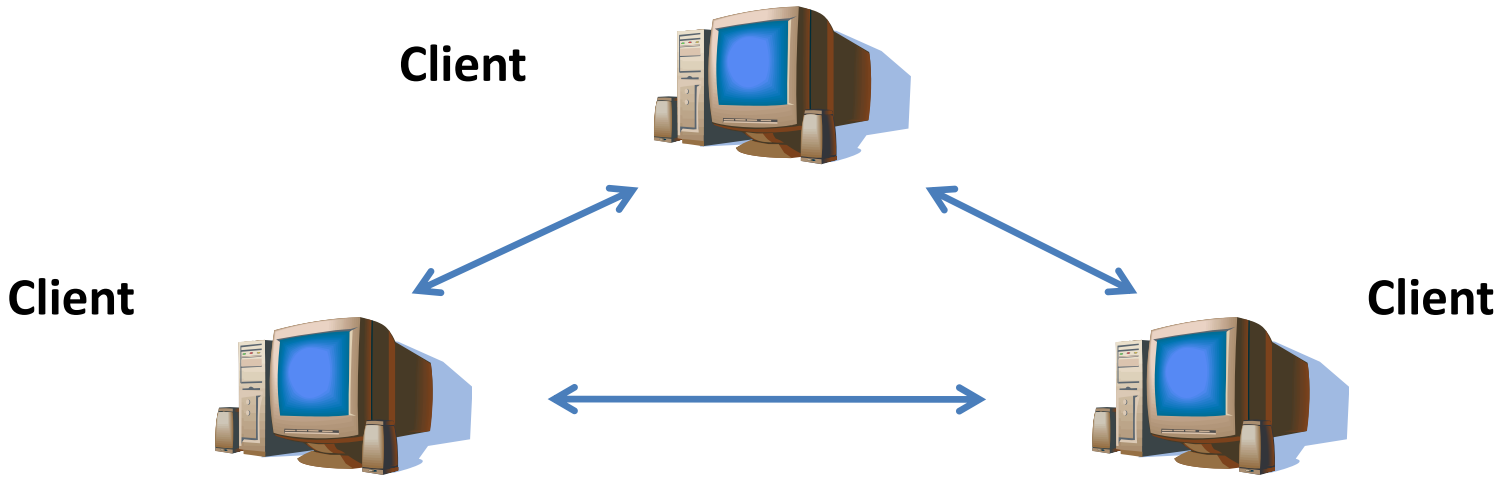
Serveur Itératif
Un seul processus effectue la réception, le traitement et l'émission



Serveur Parallèle
Le processus père effectue la réception. Il crée un fils pour réaliser le traitement et l'émission de la réponse.

Architectures réseaux

- Architecture Peer-To-Peer (P2P)
  - Architecture décentralisée
  - Pas de serveur centralisateur
  - Chaque programme client intègre partie serveur pour communication inter-clients (plus complexe)



Stockage informations

- Constructeurs peuvent utiliser des façons différentes pour stocker l'information
  - Ordre différent pour stocker octets
  - Big-Endian : en partant octet poids fort
    - Stockage de « a42b » : a4 puis 2b
  - Little-Endian : en partant octet poids faible
    - Stockage de « a42b » : 2b puis a4
  - Ex : Intel -> Little-Endian, Motorola -> Big-Endian
- Besoin de choisir un ordre d'échange des données pour communications réseaux (*Network Byte Order*)
  - Utilisation de primitives faisant la conversion

Stockage informations

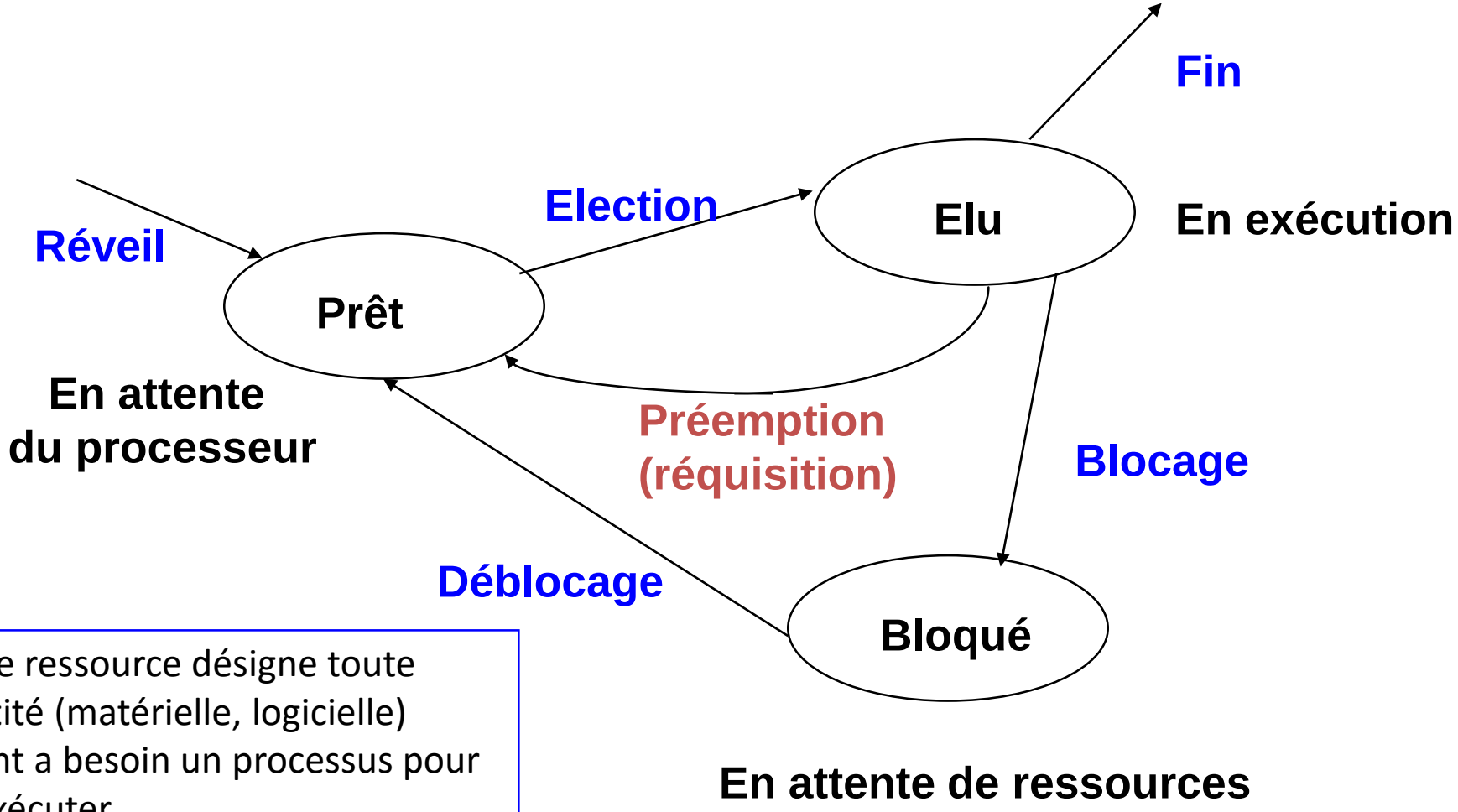
- Primitives de conversion
  - htons() : **h**ost **to** **n**etwork **s**hort
  - htonl() : **h**ost **to** **n**etwork **l**ong
  - ntohs() : **n**etwork **to** **h**ost **s**hort
  - ntohl() : **n**etwork **to** **h**ost **l**ong

Rappels système

Notion de processus

- Définition
  - Un processus est un programme en cours d ’exécution auquel est associé un environnement processeur (CO, PSW, registres généraux) et un environnement mémoire appelés contexte du processus.
  - Un processus est l ’instance dynamique d ’un programme et incarne le fil d’exécution de celui-ci
  - Un processus évolue dans un espace d’adressage protégé
  - Un processus Unix/Linux est identifié par un numéro unique, le **PID**.

Système multi-processus : Etats des processus



Une ressource désigne toute entité (matérielle, logicielle) dont a besoin un processus pour s'exécuter
(processeur, variable)

Bloc de contrôle de processus

identificateur processus
état du processus
compteur instructions
contexte pour reprise (registres et pointeurs, piles,..)
pointeurs sur file d' attente et priorité (ordonnancement)
informations mémoire (limites et tables pages/segments)
informations de comptabilisation et sur les E/S, périphériques alloués, fichiers ouverts,..

Bloc de contrôle de processus ou PCB

Primitives systèmes

- Primitives getpid, getppid

```
#include <unistd.h>

pid_t  getpid(void)
```

retourne le pid du processus appelant

```
pid_t  getppid(void)
```

retourne le pid du père du processus appelant

Primitive de création de processus

Primitive fork :

```
#include <unistd.h>
pid_t  fork(void)
```

- La primitive fork() permet la création dynamique d'un nouveau processus qui s'exécute de manière concurrente avec le processus qui l'a créé.
- Tout processus Unix/Linux hormis le processus 0 est crée à l'aide de cette primitive.
- Le processus créateur (le père) par un appel à la primitive fork() crée un processus fils qui est une copie exacte de lui-même (code et données)

Primitive de création de processus

MODE UTILISATEUR

PROCESSUS PID 12222

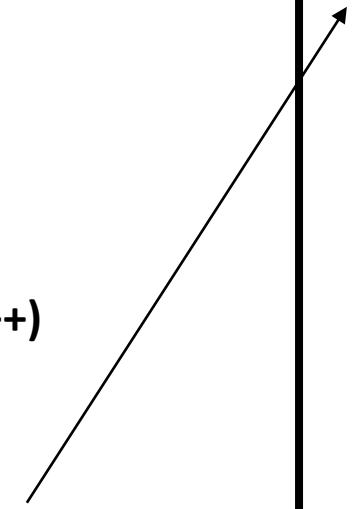
```
Main ()
{
pid_t ret ;
int i, j;

for(i=0; i<8; i++)
        i = i + j;

ret = fork();




}
```



MODE SYSTEME

Exécution de l ’appel système fork

Si les ressources noyau sont disponibles

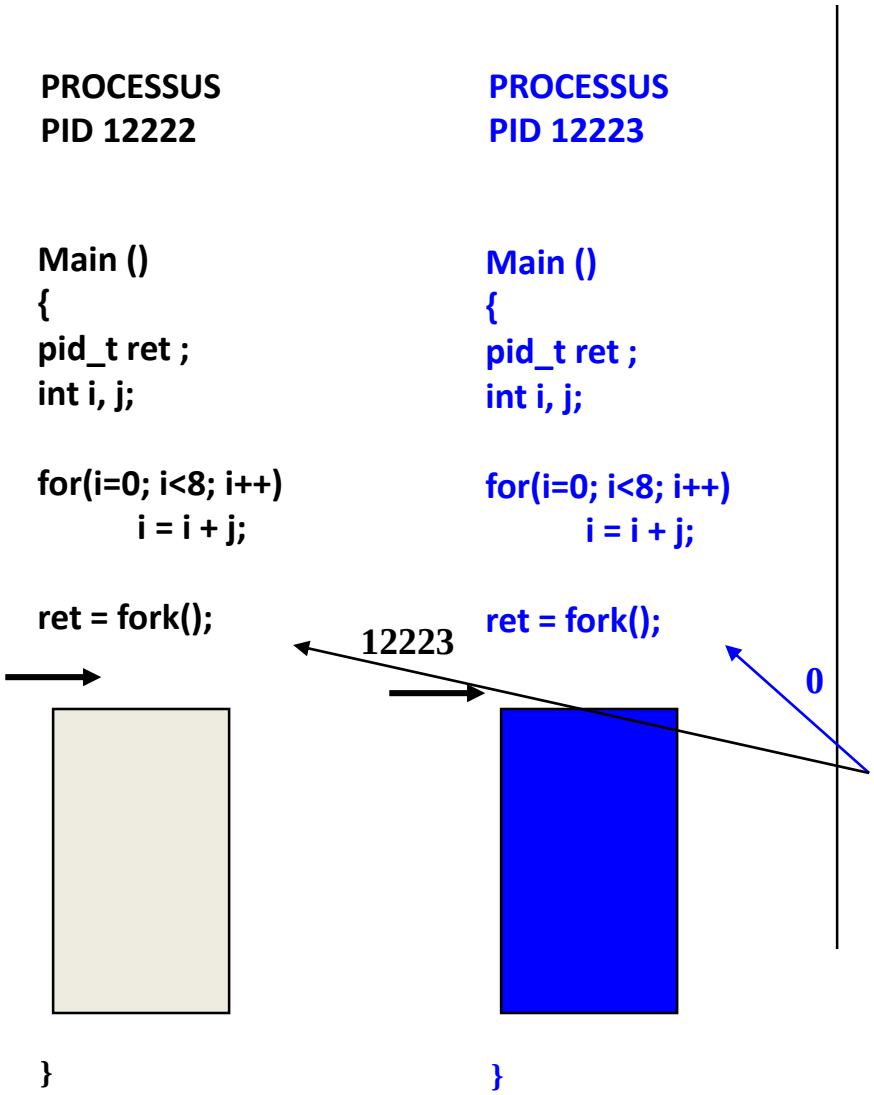
- allouer une entrée de la table des processus au nouveau processus

- allouer un pid unique au nouveau processus

- dupliquer le contexte du processus parent (code, données, pile)

- retourner le pid du processus crée à son père et 0 au processus fils

Primitive de création de processus



- Chaque processus père et fils reprend son exécution après le fork()

- Le code et les données étant strictement identiques, il est nécessaire de disposer d 'un mécanisme pour différencier le comportement des deux processus après le fork()

On utilise pour cela le code retour du fork() qui est différent chez le fils (toujours 0) et le père (pid du fils crée)

Primitive de création de processus

PROCESSUS
PID 12222

```
Main ()
{
pid_t ret ;
int i, j;

for(i=0; i<8; i++)
        i = i + j;

ret= fork();

if (ret == 0)
  printf(" je suis le fils ")
else
{
    printf ("je suis le père");
    printf ("pid de mon fils %d", ret);
 }
}
```

Pid du fils : 12223
getpid : 12222
getppid :shell

PROCESSUS
PID 12223

```
Main ()
{
pid_t ret ;
int i, j;

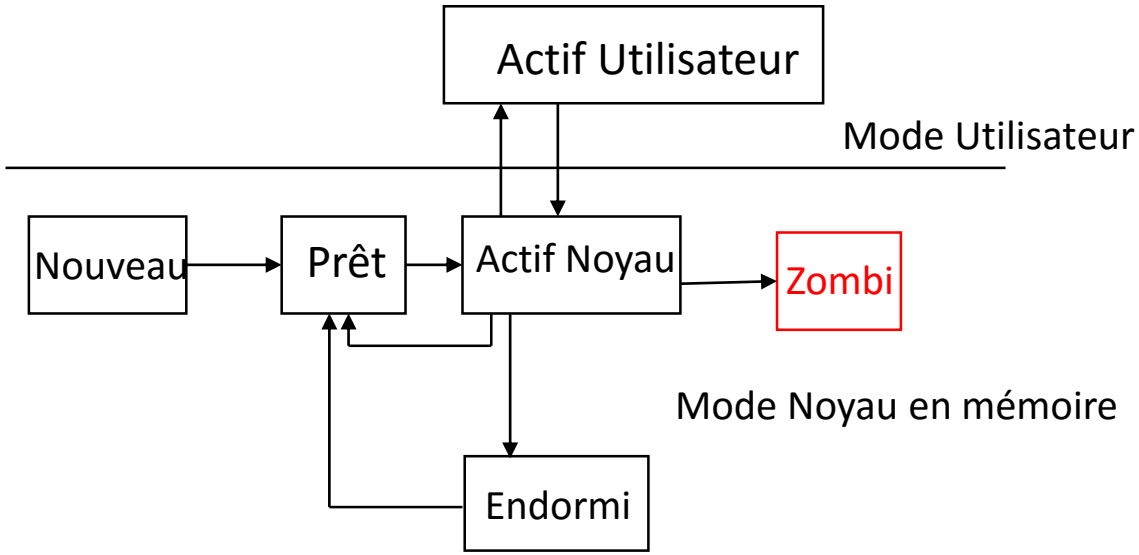
for(i=0; i<8; i++)
        i = i + j;

ret= fork();

if (ret == 0)
  printf(" je suis le fils ")
else
{
    printf ("je suis le père");
    printf ("pid de mon fils, %d" , ret);
 }
}
```

getpid : 12223
getppid :12222

Synchronisation père / fils



Primitive exit()

```
#include <stdlib.h>
void  exit (int valeur);
pid_t  wait (int *status);
```

- un appel à la primitive exit() provoque la terminaison du processus effectuant l'appel avec un code retour *valeur*. (Par défaut, le franchissement de la dernière } d'un programme C tient lieu d'exit)
- un processus qui se termine passe dans l'état Zombie et reste dans cet état tant que son père n'a pas pris en compte sa terminaison
- Le processus père "récupère" la terminaison de ses fils par un appel à la primitive wait ()

Primitive de création de processus

PROCESSUS
PID 12222

```
Main ()
{
pid_t ret ;
int i, j;

for(i=0; i<8; i++)
        i = i + j;
ret= fork();
if (ret == 0)
{
      printf(" je suis le fils ");
      exit(0); }
else
{
    printf ("je suis le père");
    printf ("pid de mon fils, %d" , ret)
    wait();
 }
}
```

PROCESSUS
PID 12223

```
Main ()
{
pid_t ret;
int i, j;

for(i=0; i<8; i++)
        i = i + j;
ret= fork();

if (ret== 0)
{
      printf(" je suis le fils ");
      exit(0); }
else
{
    printf ("je suis le père");
    printf ("pid de mon fils, %d" ,ret);
    wait();
 } }
```

Synchronisation père / fils

- Lorsqu'un processus se termine (exit), le système démantèle tout son contexte, sauf l'entrée de la table des processus le concernant.

- Le processus père, par un wait(), "récupère" la mort de son fils, cumule les statistiques de celui-ci avec les siennes et détruit l'entrée de la table des processus concernant son fils défunt.

  Le processus fils disparaît complètement.

- La communication entre le fils zombie et le père s'effectue par le biais d'un signal transmis du fils vers le père (signal SIGCHLD ou mort du fils)

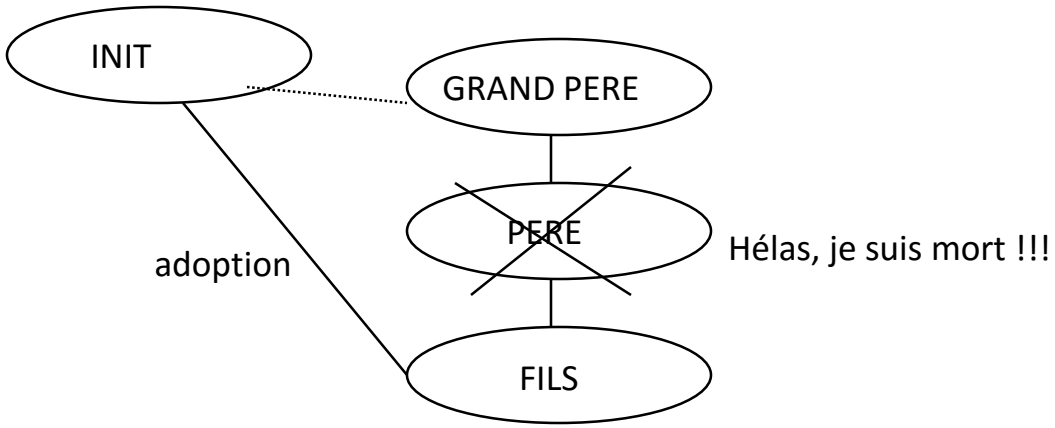
- ATTENTION : mauvaise synchronisation = saturation table des processus = blocage du système

Génétique des processus Unix

Décès et adoption

- Un processus fils défunt reste zombie jusqu'à ce que son père ait pris connaissance de sa mort

- Un processus fils orphelin, suite au décès de son père (le processus père s'est terminé avant son fils) est toujours adopté par le processus 1 (init).

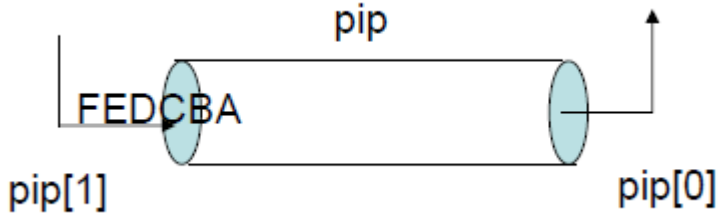


Communication entre processus

- **Communication entre processus d'une même machine**
  - **Les tubes anonymes**
  - **Les files de messages**
  - **Les segments de mémoires partagées**
- **Communication entre processus distants**
  - **Les sockets**

Communication entre processus : tubes anonymes

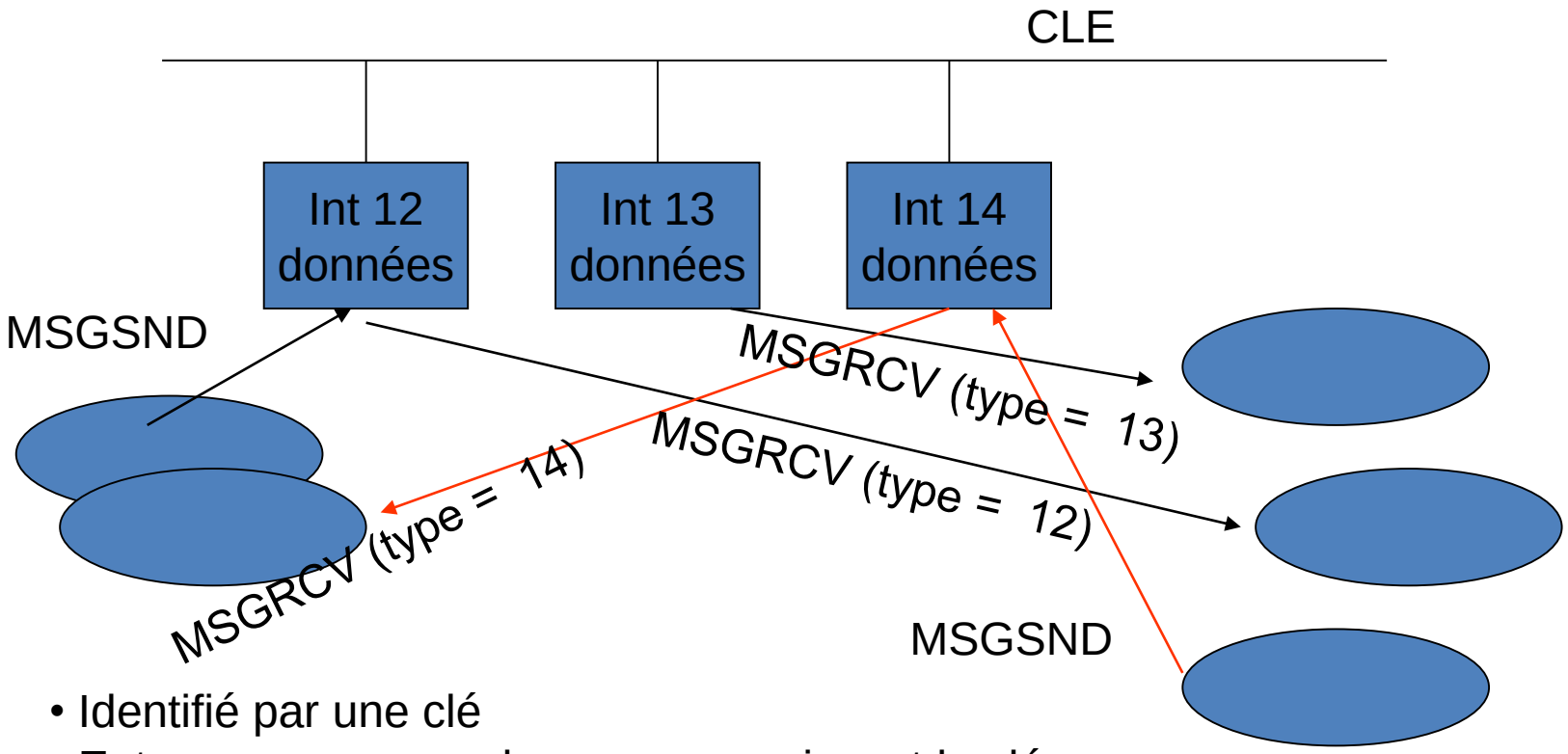
```
main ()  {
            int pip[2];
            pid_t retour;
            char chaine[7];
pipe(pip);   -- création de tube
pid = fork();   -- création fils
if (retour == 0)
            { -- le fils ecrit dans le tube
            close (pip[0]); -- pas de lecture sur le tube
            write (pip[1], "bonjour", 7);
            close (pip[1]);
            }
else
            { -- le pere lit le tube
            close (pip[1]); -- pas d'ecriture sur le tube
            read(pip[0], chaine, 7);
            close (pip[0]);
            }
```



- Identifié et accessible par les descripteurs en lecture et écriture (création, héritage)
- Entre processus de même famille
- Pas de conservation des structures
- Unidirectionnel, FIFO, lecture destructive

- Erreur : écriture dans un tube sans lecteurs

Communication entre processus : files de messages



- Identifié par une clé
- Entre processus quelconque connaissant le clé
- Bidirectionnel
- Conservation des structures
- Multiplexage

Communication entre processus : files de messages

PROCESSUS A

```
#include  <sys/types.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#define cle 17
struct msgbuf_exo {long mtype;
                   char mtext[20]; }
struct msgbuf_exo msgp;
main ()  {
int msq id ; /* identifiant de la MSQ */

/* allocation de la msq */
msqid = msgget (cle, IPC_CREAT | IPC_EXCL |
      0666);

/* ecriture message dans la msq */
msgp.mtype  =12;   on associe un type au
      message

strcpy(msgp.mtext, "ceci est un message");

msgsnd (msqid, &msgp, strlen(msgp.mtext), 0);
}
```

PROCESSUS B

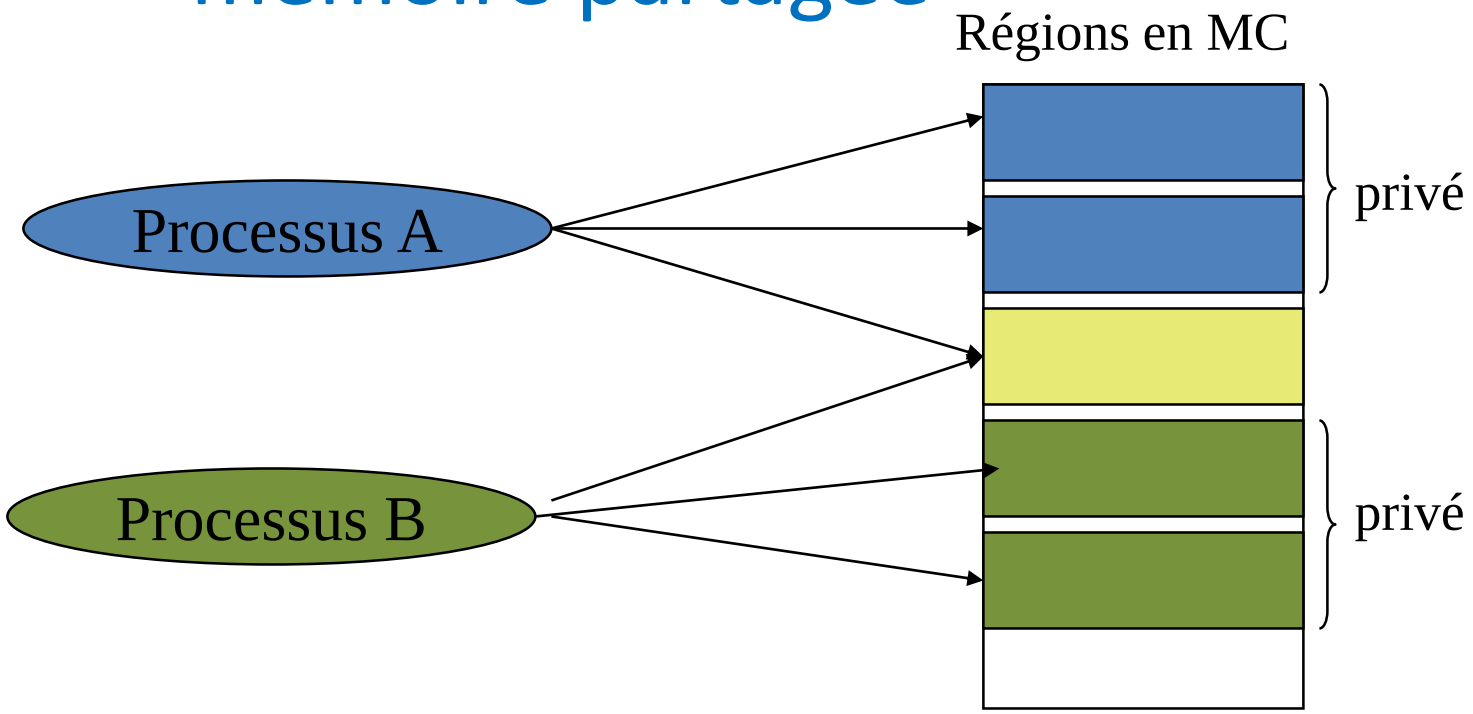
```
#include  <sys/types.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#define cle 17
struct msgbuf_exo {long type;
                   char mtext[20]; }
struct msgbuf_exo msgp;
main ()  {
int msqid ; /* identifiant de la MSQ */

/* recuperation de la msq */
msqid = msgget (cle, 0);

/* lecture message dans la msq de type 12*/
msgrcv (msqid, &msgp, 19, 12, 0);

/* destruction de la msq */
msgctl (msqid, IPC_RMID, NULL);
}
```

Communication entre processus : mémoire partagée



- Région de mémoire pouvant être partagée entre plusieurs processus
- Un processus doit attacher la région à son espace d'adressage avant de pouvoir l'utiliser
- Outil IPC repéré par une clé unique
- L'accès aux données présentes dans la région peut requérir une synchronisation (outil sémaphore)

Primitives de recouvrement

- Il s'agit d'un ensemble de primitives (famille exec) permettant à un processus de charger en mémoire, un nouveau code exécutable

- Famille de primitives exec
  - execl, execlp, execle,
  - execv, execvp, execve.

Primitives de recouvrement

int main (int argc, char *argv[], char *arge[]);

- argc est le nombre de composants de la commande
- argv est un tableau de pointeurs de caractères donnant accès aux différentes composantes de la commande
- arge est un tableau de pointeurs de caractères donnant accès à l'environnement du processus.

Sh ---> fork puis exec(calcul, 3, 4)



% calcul 3 4

on a argc = 3, argv[0]="calcul",argv[1]="3"

et argv[2] = "4"

Calcul.c

```
Main(argc,argv)
{
int somme;
if (argc <> 3) {printf("erreur"); exit();}
somme = atoi(argv[1]) + atoi(argv[2]);
exit();
}
```

ATOI() : conversion caractère --> entier

Primitives de recouvrement (execl)

```
#include <sys/types.h>
#include <sys/wait.h>
int execl(const char *ref, const char *arg, …, NULL)
```

Chemin absolu du code exécutable

arguments

PROCESSUS

```
Main ()
{
pid_t pid ;
int i, j;

for(i=0; i<8; i++)
        i = i + j;

pid= fork();

if (pid == 0)
{
      printf(" je suis le fils ");
      execl("/home/calcul","calcul","3","4", NULL);
                  executable        paramètres
}
else
{
    printf ("je suis le père");
    printf ("pid de mon fils, %d" , pid);
    wait();
 }
}
```

Création d'un processus fils par duplication du code et données du père

Le processus fils recouvre le code et les données hérités du père par ceux du programme calcul.
Le père transmet des données de son environnement vers son fils par les paramètres de l'exec

Le père attend son fils

Primitives de recouvrement

Il y a six primitives qui diffèrent :

- par la manière dont les arguments argv sont passés
  - liste (execl, execlp, execle)
  - tableau (execv, execvp, execve)
- par la manière dont le chemin de l'exécutable à charger est spécifié
  - en utilisant la variable d'environnement PATH (execlp, execvp)
  - relativement au répertoire de travail (les autres)
- par la modification de l'environnement (execve, execle)

Notion de processus léger

Définition

- Extension du modèle traditionnel de processus
- Un thread ou processus léger est un fil d ’exécution au sein d’un processus. On peut avoir plusieurs fils d’exécution au sein du processus.

Fil d ’exécution	Ressources	Espace d ’adressage

Processus classique

Fil d ’exécution	Ressources	Espace d ’adressage
Fil d ’exécution		
Fil d ’exécution		

Processus à threads

Notion de processus léger

Primitives

- Création

```
int pthread_create (pthread_t  *thread, const pthread_attr_t  *attr, void *
(*start_routine)(void *),  void   *arg);
```

- synchronisation entre threads

```
int   pthread_join ( pthread_t    thread, void **value_ptr);
```

- terminaison de threads

```
int   pthread_exit (void **value_ptr);
```

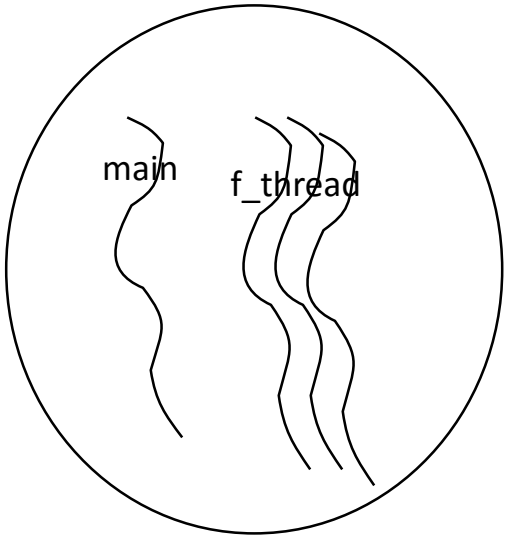
Notion de processus léger

```
#include <stdio.h>
#include <pthread.h>

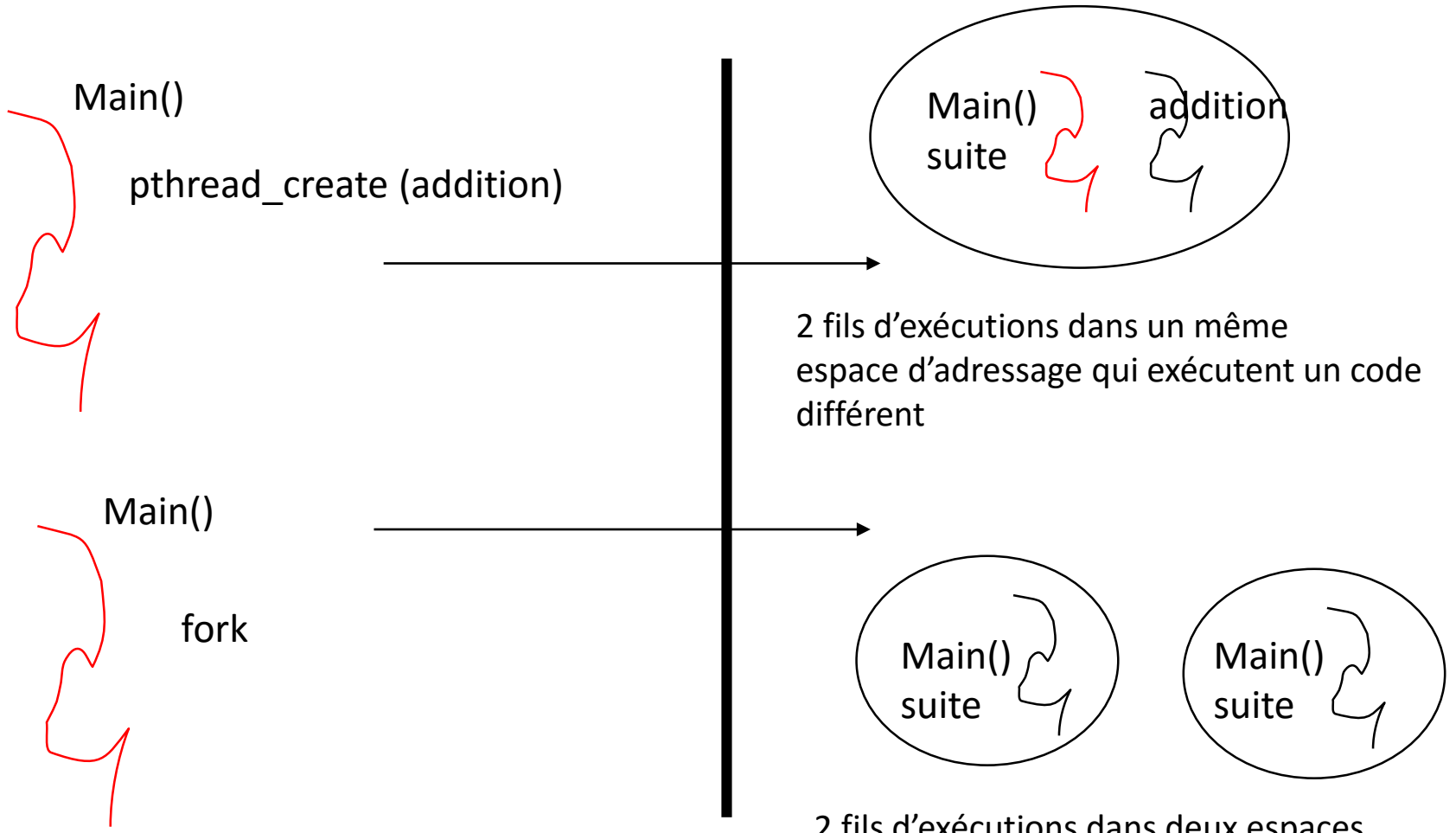
pthread_t pthread_id[3];

void f_thread(int i) {
        printf ("je suis la %d-eme pthread d'identite %d.%d\n", i, getpid(),
        pthread_self());
}

main()
{
        int i;
        for (i=0; i<3; i++)
        { if (pthread_create(pthread_id + i, NULL, f_thread, i) == -1)
              fprintf(stderr, "erreur de creation pthread numero %d\n", i);
          printf ("je suis la thread initiale %d.%d\n", getpid(), pthread_self());
        }
        pthread_join();
}
```



Notion de processus léger



Notion de processus léger

Processus classique (lourd)

espace d 'adressage protégé à un fil d 'exécution



Commutation de contexte
toujours changement d'espace d'adressage

Communication
outils entre espace d'adressage (tubes, messages queues)

Pas de parallélisme dans un espace d 'adressage

Processus à threads (léger)

espace d 'adressage protégé à n fils d 'exécution (n ≥ 1)



Commutation de contexte : allégée
pas de changement d'espace d'adressage entre fils d'un même processus

Communication : allégée
les fils d'exécution d'un même processus partagent les données, mais attention à la synchronisation

Parallélisme dans un espace d 'adressage

Signaux : définition

- Le signal est une interruption logicielle délivrée à un processus.
  - Il informe les processus de l'occurrence d'événements asynchrones.
  - Il ne transporte pas de données.
  - Un signal est associé à un événement

- 64 signaux identifiés par un numéro et un nom (SIG*X*)
  - 1 à 31 : signaux classiques
  - 32 à 63 : signaux temps réel

- L'événement associé à un signal
  - extérieur au processus (frappe caractère, terminaison d'un autre processus)
  - intérieur au processus correspondant à une erreur (erreur arithmétique ou violation mémoire) : levée d'une trappe.

Liste des signaux selon le standard POSIX

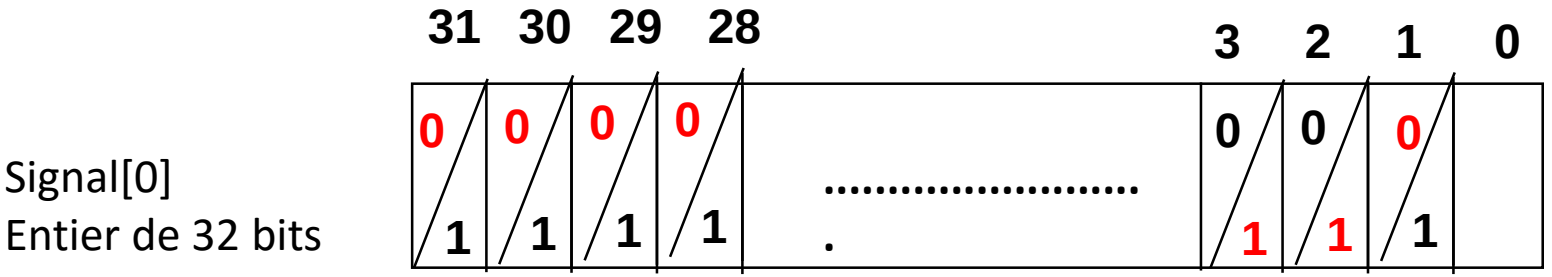
- signaux relatifs à la fin de processus :
  - SIGCHLD (17) : mort du fils
  - SIGKILL (9) : signal de terminaison
- signaux relatifs à des erreurs
  - SIGILL (4) : instruction illégale
  - SIGFPE (8) : erreur arithmétique
  - SIGSEGV (11) : violation mémoire
  - SIGPIPE (13) : écriture dans un tube sans lecteur
- signaux relatifs aux temporisations
  - SIGALRM (14): fin de temporisation (fonction alarm)

Liste des signaux selon le standard POSIX

- signaux relatifs aux interactions avec le terminal (extinction, frappe touche DELETE et BREAK)
  - SIGHUP, SIGINT (crtl C), SIGQUIT (crtl \)
- signaux relatifs à la mise au point de programmes
- Deux signaux disponibles pour les utilisateurs (SIGUSR1, SIGUSR2)

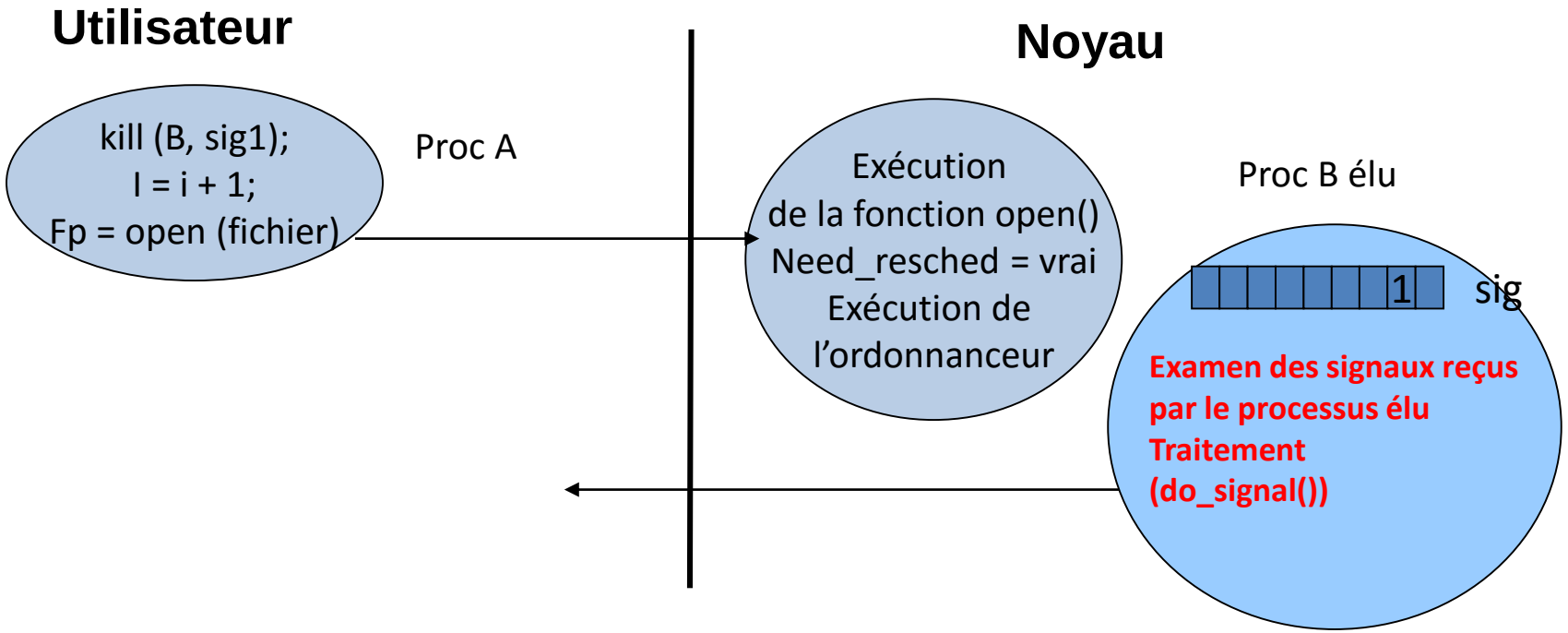
Envoi d'un signal à un processus

- Un processus A envoie un signal N à un processus B par le biais d'une fonction kill.
  - kill (pid B, N°signal)
  - Positionnement d'un bit à 1 dans le champ *signal* de la table des processus, correspondant au type de signal reçu.
  - signal pendant (reçu)
  - vecteur de bits : pas de mémorisation du nombre de signaux d'un type reçu.



Prise en compte d'un signal reçu par le processus B

- Le processus B examine et traite les signaux reçus chaque fois qu'il quitte le mode noyau pour retourner au mode utilisateur
  - signal délivré ; le bit correspondant est remis à 0



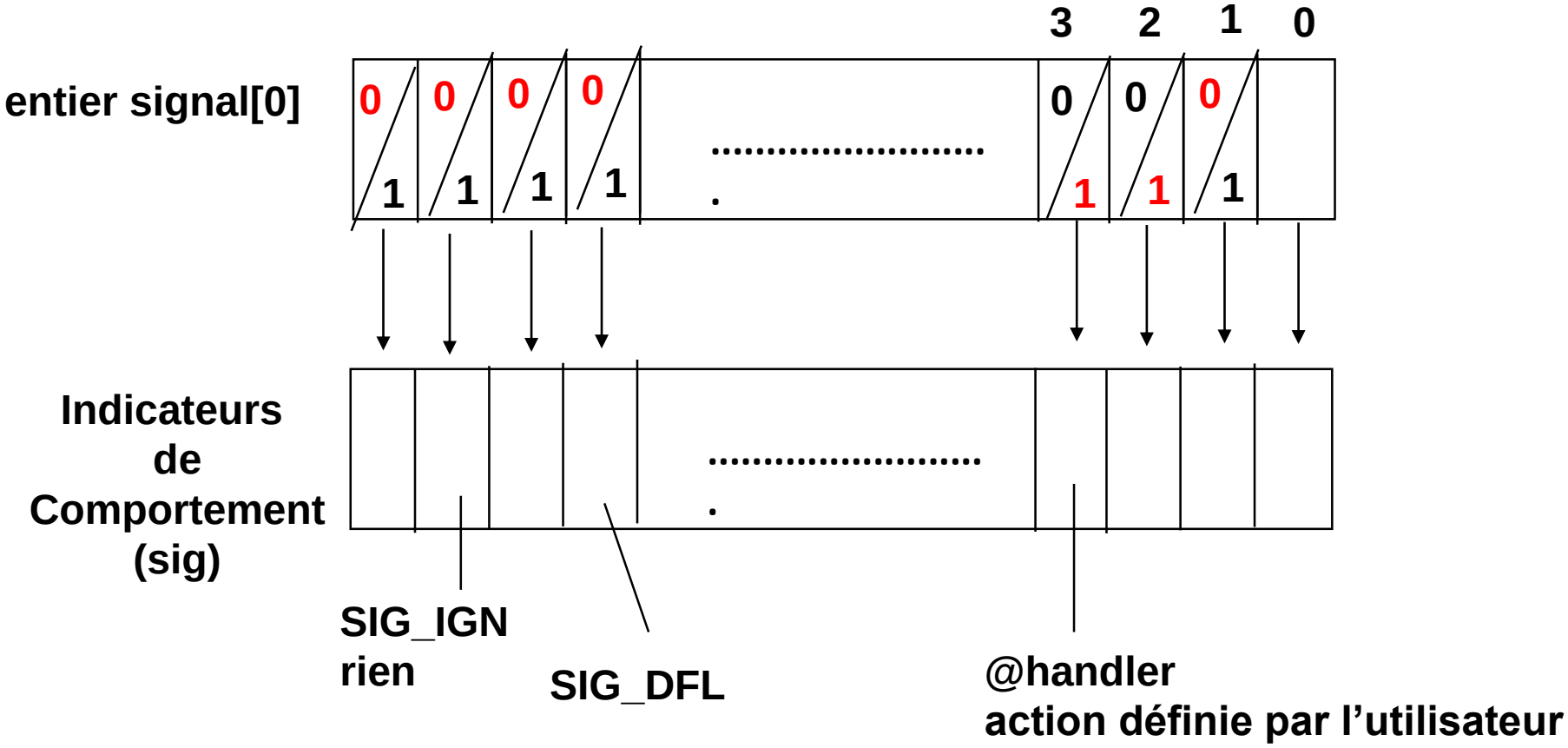
Comment un processus contrôle-t-il ses réactions vis-à-vis des signaux ?

Traitement des signaux

- A tout signal est associé un traitement par défaut (ignorer, terminer le processus avec ou sans code, stopper le processus);
- Tout processus peut installer pour chaque type de signal (hormis SIGKILL), un nouveau handler
  - handler SIG_IGN : pour ignorer le signal (sauf mort du fils pour Linux)
  - handler fonction utilisateur pour capter le signal
    - ↳ fonction signal et sigaction

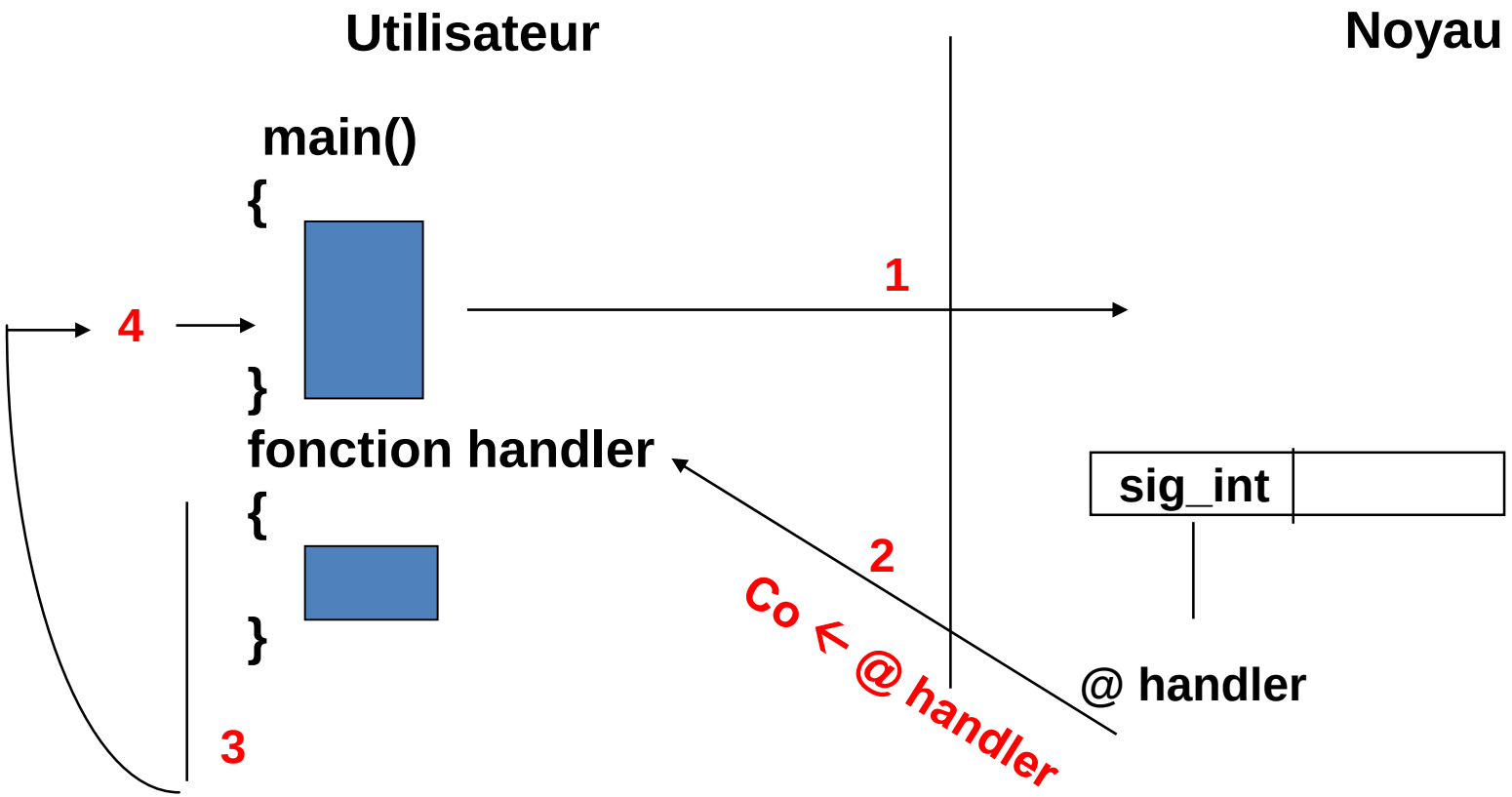
Comment un processus contrôle-t-il ses réactions vis-à-vis des signaux ? (fonction du noyau do_signal())

Traitement des signaux



Exécution d'un handler de signal défini par l'utilisateur

Traitement des signaux



Les fonctions liées aux signaux

- Envoyer un signal à un processus
  - int kill (pid_t pid, int sig) kill (12563, SIGKILL)
  - kill -n°signal pid kill -9 12563
- Associer un handler à un signal
  - signal(int sig, fonction) signal(SIGINT, p_hand)
  - sigaction(int sig, struct sigaction action, struct sigaction old_action)
- Armer une temporisation
  - int alarm (int seconds) alarm(10)
  - au bout de *seconds* unités de temps, le signal SIGALRM est envoyé au processus
- Attendre un signal
  - int pause();

Primitive sigaction()

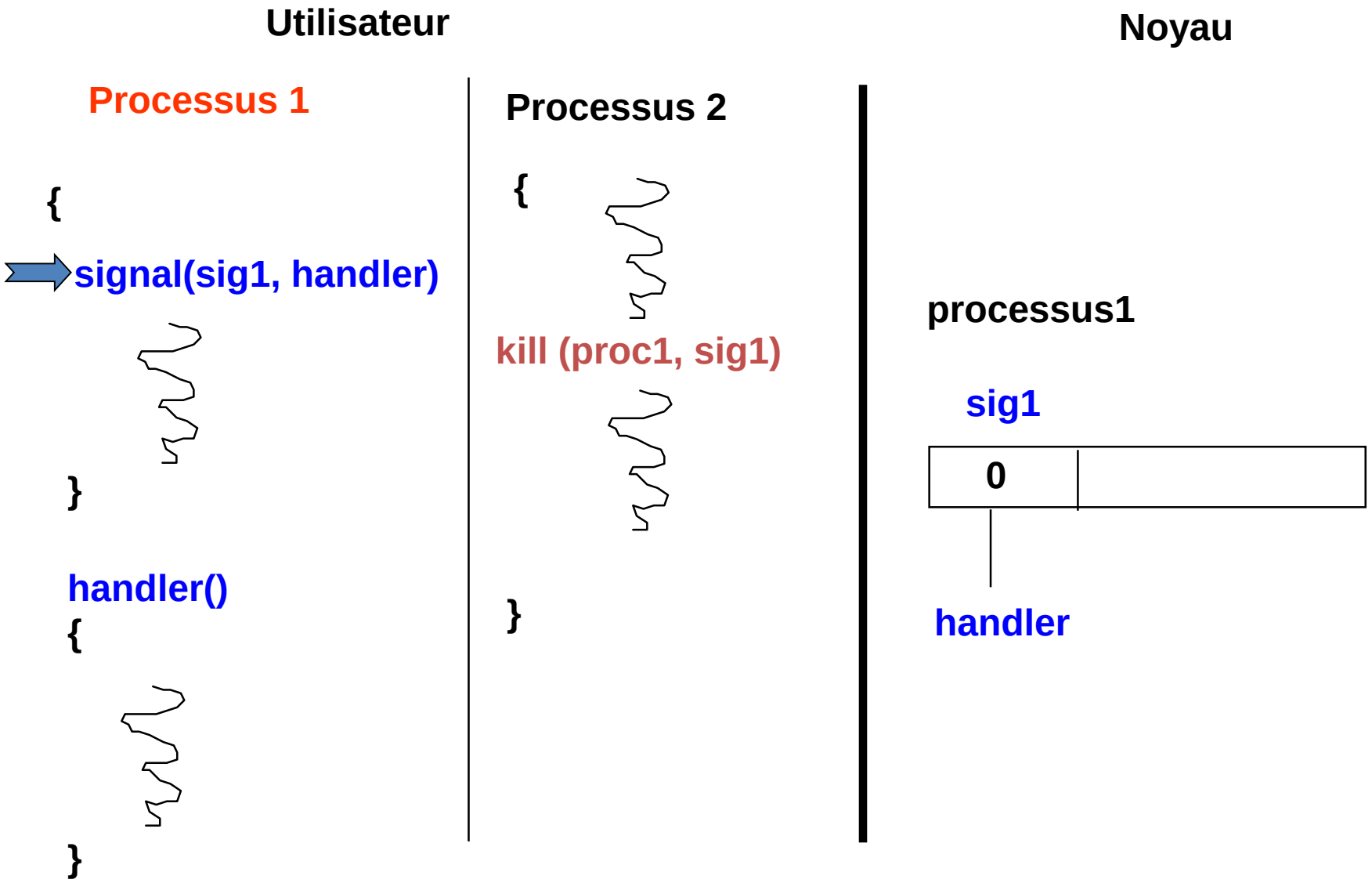
Associer un handler à un signal (POSIX)

```
int sigaction (int sig, const struct sigaction *p_action,  struct sigaction *p_action_anc)

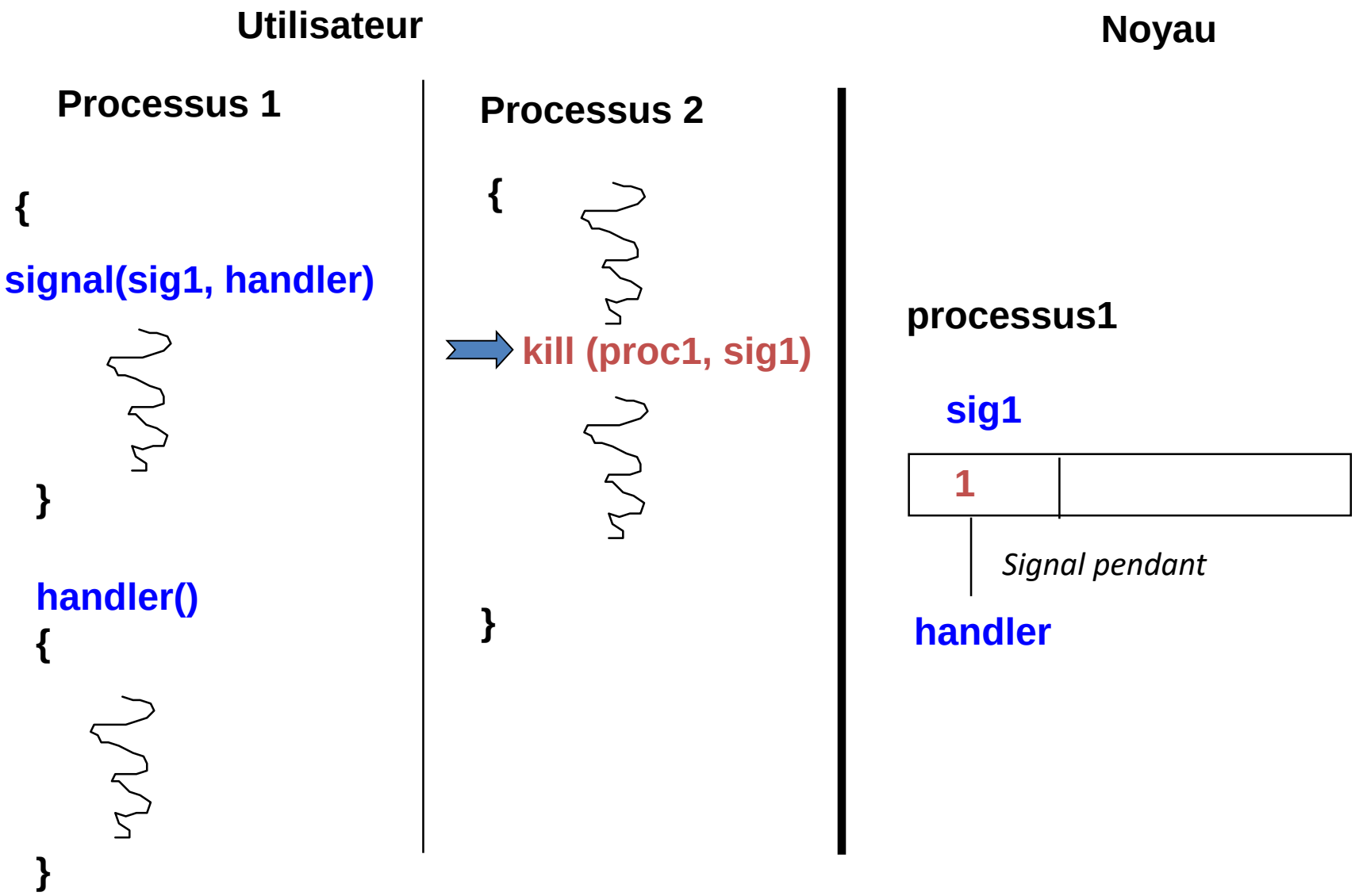
struct sigaction {void (*sa_handler)();   /* SIG_DFL ou SIG_IGN ou pointeur sur handler */
                  sigset_t sa_mask;    /* masque sur signaux */
                  int sa_flags; }         /* options… à mettre à NULL */


struct sigaction action;
action.sa_handler = p_hand;
sigaction (SIG_INT, &action, NULL);
```

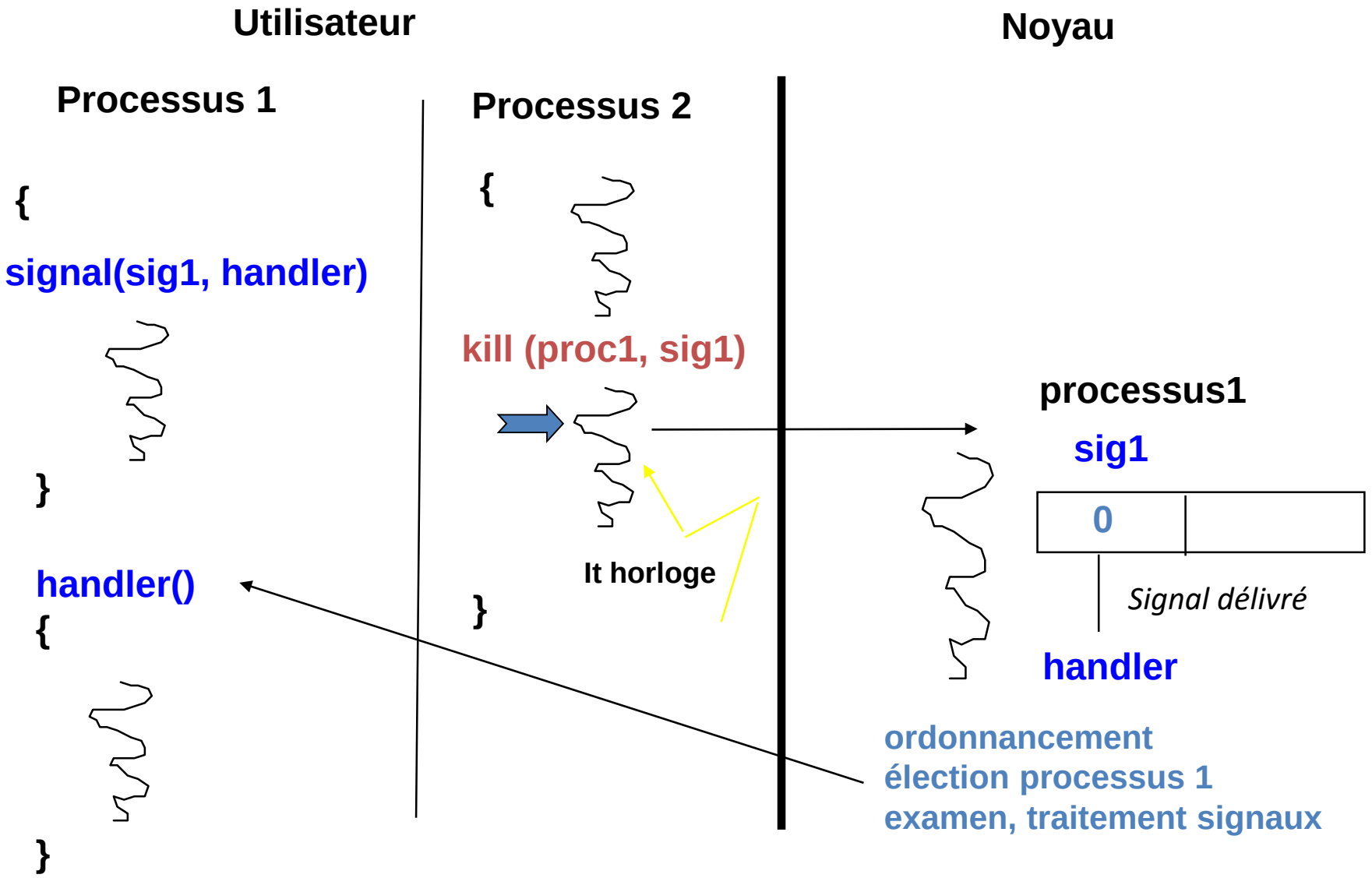
Signaux : synthèse



Signaux : synthèse



Signaux : synthèse



Signaux et interruptions

- Signaux

➢ Le processus P2 envoie un signal au processus P1 (signal pendant chez P1)

➢ Plus tard, le processus P1 est élu. Il quitte le mode noyau. Il exécute le handler du signal en mode utilisateur (signal délivré à P1).

- Interruptions

➢ Le dispositif matériel X envoie une interruption lors de l'exécution du processus P1.

➢ Immédiatement, le processus P1 est dérouté en mode noyau pour exécuter le handler « routine » de l'interruption.

Signaux et interruptions

/* programme où un processus crée un fichier sur lequel il travaille. En cas de réception du signal SIGINT, le processus ferme le fichier sur lequel il travaille avant de se terminer */

```
#include <sys/types.h>
#include <signal.h>
#include <unistd.h>
#include <stdio.h>
#include <fcntl.h>

int desc;

main()
{
int i;
 extern onintr();    /* Handler */

desc = open("toto", O_RDWR, 0);
signal (SIGINT, onintr);
i = 0;
while (i < 5)
{               write (desc, "abcdefghijgl", 12);
                i = i + 1;
}
printf("fin process normal\n");
close (desc);
}

onintr ()
{
                printf("Handler onintr\n");
                close(desc);
                exit ();
}
```

Signaux et interruptions

/* programme où un processus crée un fils puis se met en attente de la fin de son fils. Le fils exécute un code qui boucle. Au bout de 10 secondes, le fils n'étant pas achevé, le père tue son fils */

```
#include <sys/types.h>
#include <unistd.h>
#include <stdio.h>
#include <signal.h>
 pid_t pid;

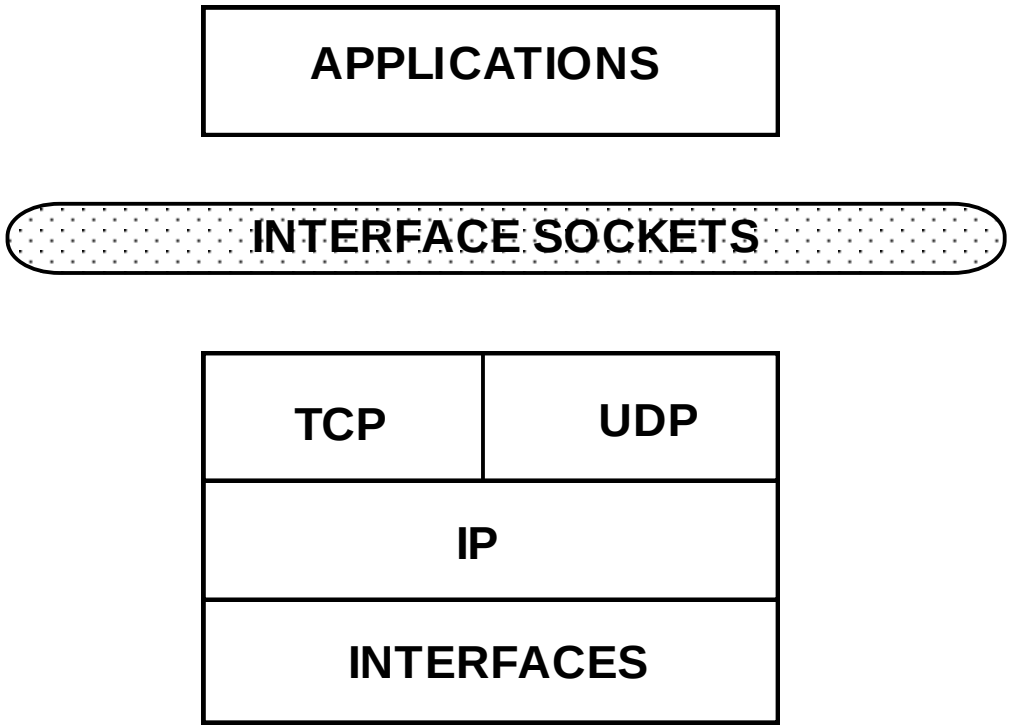
main()
{
extern onalarm();    /* Handler */

pid = fork();
if (pid == -1)
    printf ("erreur creation de processus");
else
{       if (pid == 0)
        {
            printf ("valeur du fork, %d", pid);
            printf (" je suis le fils, mon pid est %d\n", getpid());
            for(;;)
               printf("je boucle !!!! \n");
            exit();   }
         else  {
             printf ("valeur du fork, %d", pid);
             signal (SIGALRM, onalarm);
              alarm(5);
              wait ();
               exit(); }
}

onalarm ()
{               printf("Handler onalarm\n");
                kill (pid, SIGKILL);
}
```

Communication par Sockets

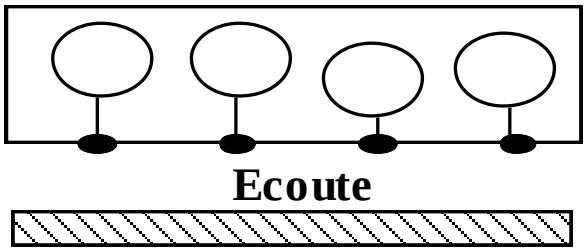
L'interface socket : situation



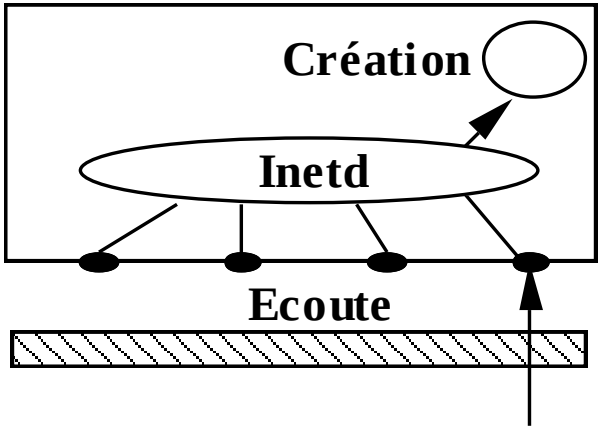
- Interface de programmation (ensemble de primitives)
- Point de communication (adresse d'application)
- Compatible SGF

L'interconnexion de réseau : quelques principes

La couche Application



Un démon par service
=> Encombrement de la table des processus



Le démon inetd
en écoute sur les différents ports crée le serveur sollicité

Qu'est ce qu'une socket

- Connexion réseau représentée par descripteur (comme pour fichier sur disque)
  - « *Tout est fichier dans un système Unix* »
  - Descripteur de fichier est un entier associé à un fichier ouvert dans le système
  - Possibilité d'utiliser même primitive que pour fichiers classiques
    - Pour avoir plus de contrôle, utilisation primitives dédiées pour communications en réseau

Types de socket

- Plusieurs types de socket
- 3 trois types principaux
  - Stream sockets : communication connectée
    - Utilise le protocole TCP, dénommé SOCK_STREAM (http://tools.ietf.org/html/rfc793)
  - Datagram sockets : communication non-connectée
    - Utilise le protocole UDP, dénommé SOCK_DGRAM (http://tools.ietf.org/html/rfc791)
  - Raw sockets :
    - Permet d'accéder directement à la couche IP (droit super-utilisateur), dénommé SOCK_RAW
- Les deux premiers sont les plus utilisés

Structure de données pour adresses

- Définition de la structure de données stockant adresse(s) de l'hôte
  - Chaînage entre structures (via pointeur ***ai_next***)
  - Permet d'utiliser adresses IPv4 ou IPv6 (indiqué par ***ai_family***)

```
struct addrinfo {
        int ai_flags;           // AI_PASSIVE, AI_CANONNAME, etc.
        int ai_family;          // AF_INET, AF_INET6, AF_UNSPEC
        int ai_socktype;        // SOCK_STREAM, SOCK_DGRAM
        int ai_protocol;        // use 0 for "any"
        size_t ai_addrlen;      // size of ai_addr in bytes
        struct sockaddr *ai_addr; // struct sockaddr_in or _in6
        char *ai_canonname;     // full canonical hostname

        struct addrinfo *ai_next; // linked list, next node
};
```

Structure de données pour adresses

- Structure *addrinfo* est rempli en faisant appel à
  - La fonction **getaddrinfo()**, permet de ne plus remplir manuellement structure et être IPv4 ou IPv6 compatible
  - Utilisation de la première structure correcte de la liste (voir exemple)
  - Pointeur ***ai_addr*** référence structure ***sockaddr*** contenant l'adresse IP à utiliser
    - ***sa_family*** : AF_INET pour adresse IPv4, AF_INET6 pour adresse IPv6
    - ***sa_data*** : contient adresse IP et numéro de port

```
struct sockaddr {
        unsigned short sa_family;  // address family, AF_xxx
        char sa_data[14];          // 14 bytes of protocol address
};
```

- Pb : structure pas pratique à remplir manuellement -> ***sockaddr_in***

Structure de données pour adresses

- Structure ***sockaddr_in*** utilisée pour IPv4
  - Voir ***sockaddr_in6*** pour IPv6

```
struct sockaddr_in {
        short int   sin_family;          // Address family, AF_INET
        unsigned short int sin_port;     // Port number, Network Byte Order
        struct in_addr     sin_addr;     // Internet address
        unsigned char      sin_zero[8];  // Same size as struct sockaddr
};
```

  - Possibilité de passer de l'un à l'autre en faisant « cast »
  - ***sin_zero*** : complétée avec zéros via fonction **memset()**
  - ***sin_port*** : à remplir en utilisant fonction **htons()**

```
struct in_addr {
        uint32_t  s_addr;          // that's a 32-bit int (4 bytes)
};
```

Structure de données pour adresses

- Structure ***sockaddr_in6*** utilisée pour IPv6
  - Structure similaire pour adresse IPv6

```
struct sockaddr_in6 {
        u_int16_t       sin6_family;    // address family, AF_INET6
        u_int16_t       sin6_port;      // port number, Network Byte Order
        u_int32_t       sin6_flowinfo;  // IPv6 flow information
        struct in6_addr sin6_addr;      // IPv6 address
        u_int32_t       sin6_scope_id;  // Scope ID
};

struct in6_addr {
        unsigned char s6_addr[16]; // IPv6 address
};
```

Structure de données pour adresses

- Conversion adresse IP au format indiqué par ***sockaddr_in*** ou ***sockaddr_in6***

```
struct sockaddr_in sa;       // IPv4
struct sockaddr_in6 sa6;     // IPv6

inet_pton(AF_INET, "192.0.2.1", &(sa.sin_addr));                    // IPv4
inet_pton(AF_INET6, "2001:db8:63b3:1::3490", &(sa6.sin6_addr));     // IPv6
```

  - ***pton*** : pour **P**resentation **to** **N**etwork
  - Code de retour doit être supérieur à 0

Structure de données pour adresses

- Récupérer adresse IP sous forme de caractère indiqué par ***sockaddr_in*** ou ***sockaddr_in6***
  - Avec ***INET_ADDRSTRLEN*** ou ***INET6_ADDRSTRLEN*** taille max adresse IPv4 ou IPv6

```
// IPv4:
char ip4[INET_ADDRSTRLEN];         // space to hold the IPv4 string
struct sockaddr_in sa;             // pretend this is loaded with something
inet_ntop(AF_INET, &(sa.sin_addr), ip4, INET_ADDRSTRLEN);
printf("The IPv4 address is: %s\n", ip4);

// IPv6:
char ip6[INET6_ADDRSTRLEN];        // space to hold the IPv6 string
struct sockaddr_in6 sa6;           // pretend this is loaded with something
inet_ntop(AF_INET6, &(sa6.sin6_addr), ip6, INET6_ADDRSTRLEN);
printf("The address is: %s\n", ip6);
```

Changements pour compatibilité IPv6

- Utiliser fonction ***getaddrinfo()*** au lieu de ***gethostbyname()***
- Utiliser fonction ***getnameinfo()*** au lieu de ***gethostbyaddr()***

- Ajouter le chiffre « 6 » dans les macros ou types :
  - Utiliser ***AF_INET6*** au lieu de ***AF_INET***
  - Utiliser ***in6addr_any*** au lieu de ***INADDR_ANY***
  - Utiliser structure ***sockaddr_in6*** au lieu de ***sockaddr_in***
  - Utiliser ***inet_pton()*** au lieu de ***inet_aton()*** ou ***inet_addr()***
  - Utiliser ***inet_ntop()*** au lieu de ***inet_ntoa()***

Initialisation structures pour socket

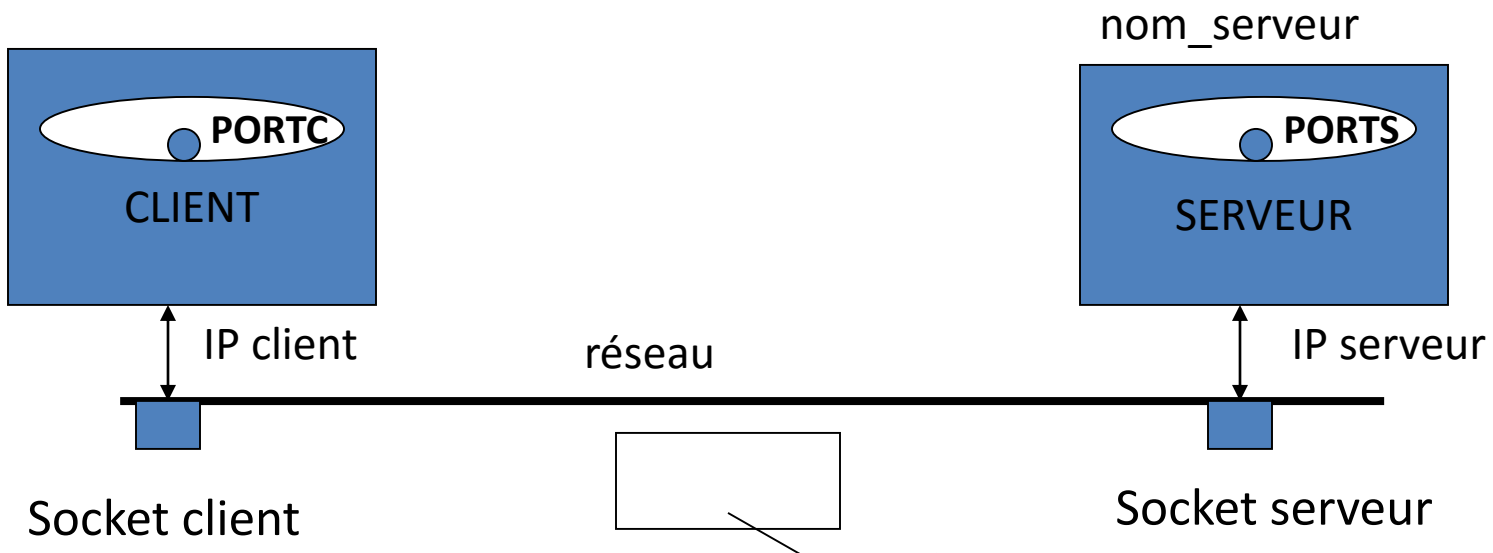
- Initialisation de la structure ***addrinfo*** :

```
#include <sys/types.h>
#include <sys/socket.h>
#include <netdb.h>

int getaddrinfo(const char *node,     // e.g. "www.example.com" or IP
                const char *service,  // e.g. "http" or port number
                const struct addrinfo *hints, // additional infos on socket type
                struct addrinfo **res);
```

- Retour : 0 si succès, erreur sinon récupérable via *gai_strerror()*
- Résultat de la fonction est retournée dans champ ***res***

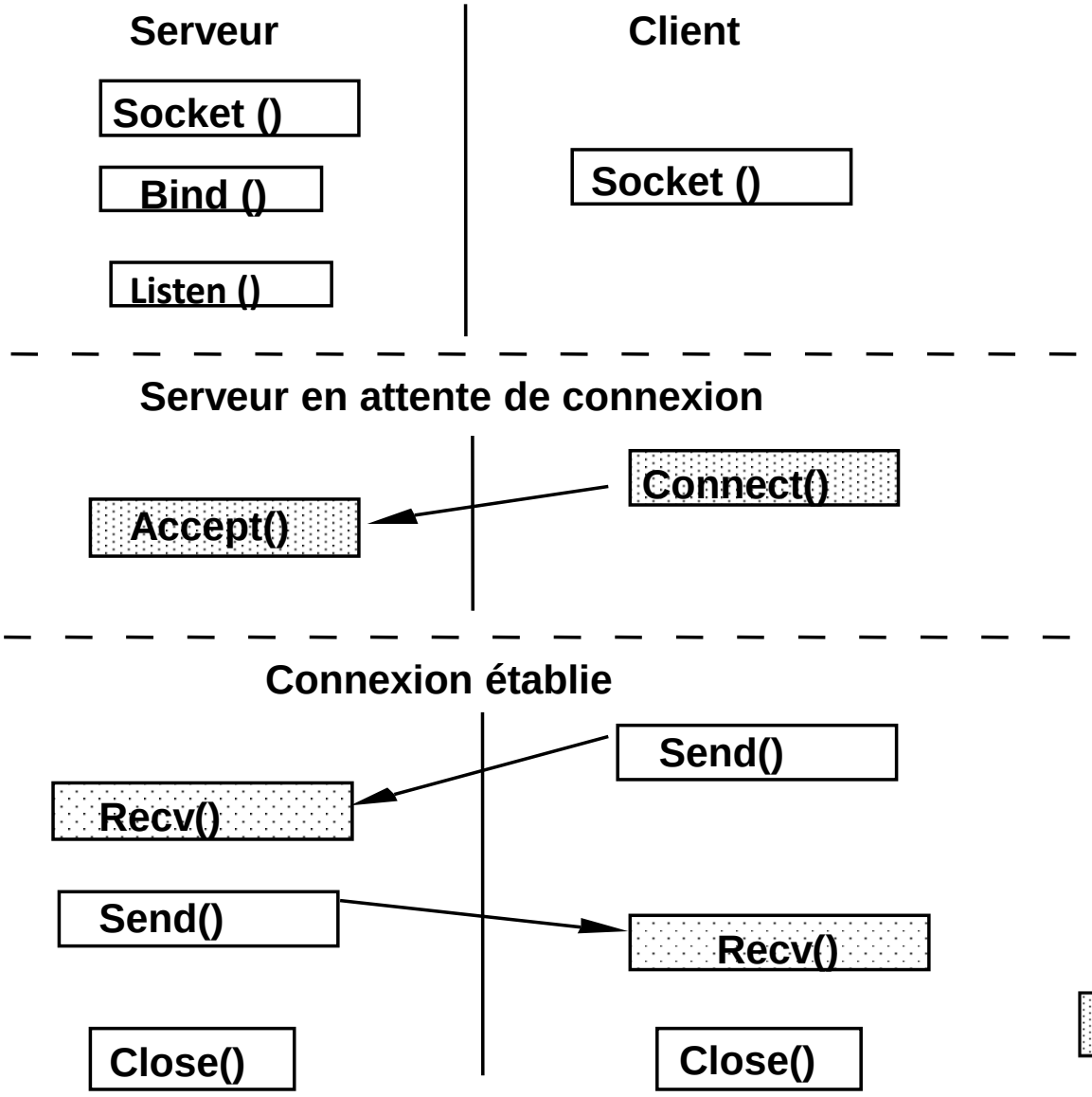
Attachement d'une adresse d'application



Adresse locale
Socket client
Port ← PORTC
Adresse IP ← INADDR_ANY ou in6addr_any

Adresse distante
Port ← PORTS
Adresse IP ← getaddrinfo(nom_serveur)

Communication en mode connecté



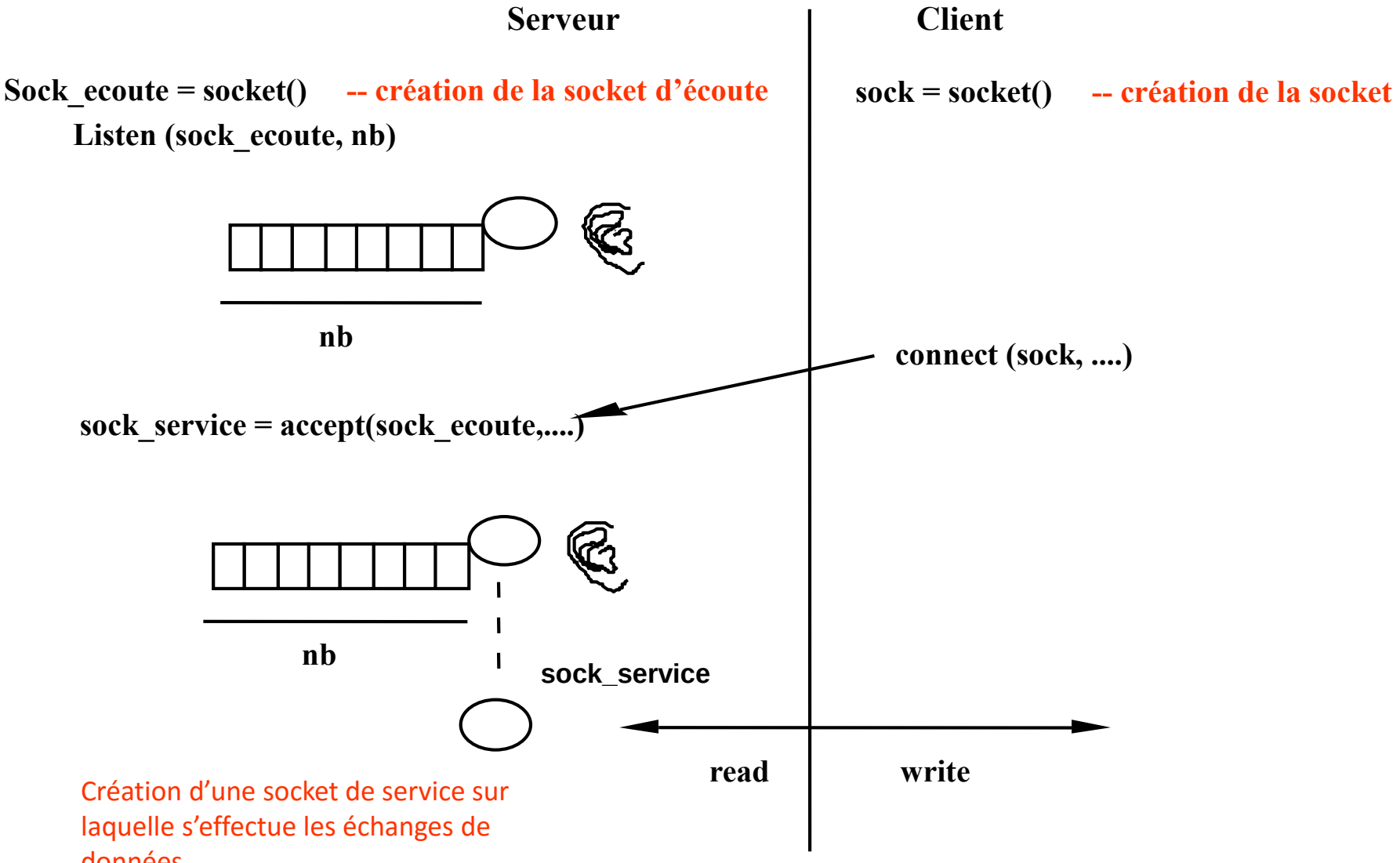
Sock = socket (AF_INET, SOCK_STREAM, IPPROTO_TCP)

Etablissement de connexion avec échange des adresses participantes

→ Les messages échangés ne contiennent pas l'adresse du destinataire

Appels bloquants

Communication en mode connecté

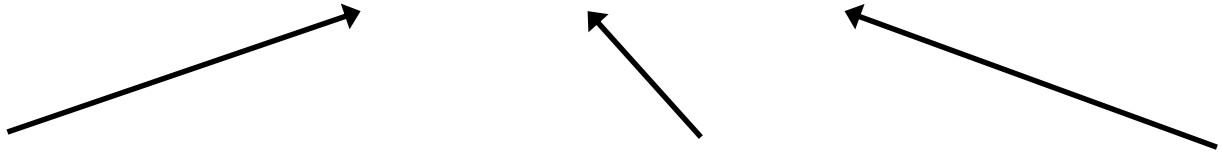


Création d'une socket de service sur laquelle s'effectue les échanges de données

Création d'une socket

```
#include <sys/types.h>
#include <sys/socket.h>

sockfd = socket(int af, int type, int protocole)
```



Famille d'adresse de la socket :
AF_INET : protocole internet
AF_UNIX : protocole interne à UNIX

Type de service :
SOCK_STREAM
SOCK_DGRAM
SOCK_RAW

IPPROTO_TCP
IPPROTO_UDP
0 par défaut

- **Retourne un descripteur de socket ayant les mêmes propriétés qu'un descripteur de fichier (héritage) accessible par le créateur et les fils de celui-ci**
- **Retourne -1 si erreur**

Exemple création d'une socket

```
#define PORT "3490"

int status,sockfd;
struct addrinfo hints;
struct addrinfo *res;

memset(&hints, 0, sizeof(hints)); // make sure the struct is empty
hints.ai_family = AF_UNSPEC;      // don't care IPv4 or IPv6, otherwise use AF_INET or AF_INET6
hints.ai_socktype = SOCK_STREAM; // TCP stream sockets
hints.ai_flags = AI_PASSIVE;      // fill in my IP, or use IP address (first parameter of getaddrinfo())

status = getaddrinfo(NULL, PORT, &hints, &res);

if (status!=0)
{               fprintf(stderr, "getaddrinfo error: %s\n", gai_strerror(status));
                exit(1);}

sockfd = socket(res->ai_family, res->ai_socktype, res->ai_protocol);
```

Informations sur extrémités socket

```
#include <sys/socket.h>
int getpeername(int sockfd, struct sockaddr *addr, int *addrlen);
```



Socket d'écoute côté serveur

Structure avec adresse et port rétournés

Taille structure adresse (en octets)

- Retourne l'adresse et port nœud connecté à la socket
- Retourne 0 si succès, -1 sinon

```
#include <unistd.h>
int gethostname(char *hostname, size_t size);
```



Adresse buffer où stocker nom de l'hôte

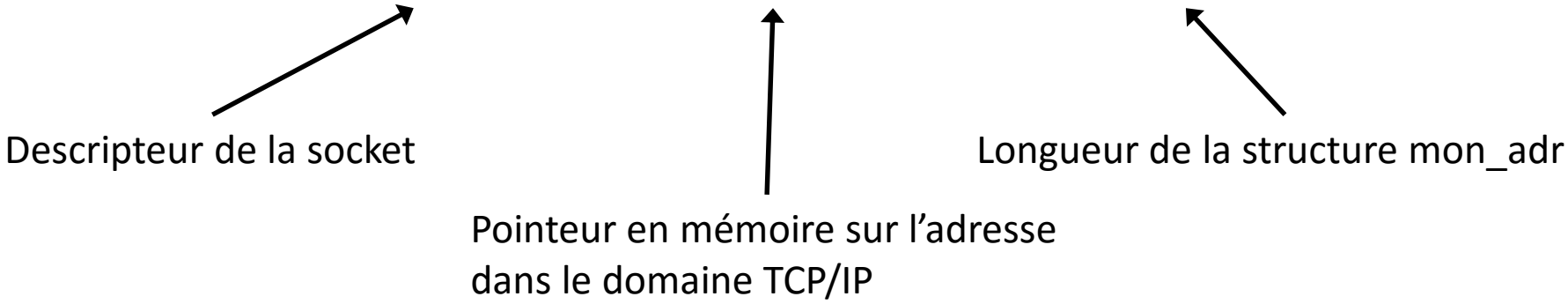
Taille de la chaîne de caractère

- Retourne le nom de la machine hôte exécutant le programme
- Retourne 0 si succès, -1 sinon

Attachement d'une adresse d'application

```
#include <sys/types.h>
#include <sys/socket.h>

int bind(int sockfd, struct sockaddr *mon_adr, int addrlen);
```



- L'opération d'attachement permet d'étendre le groupe des processus pouvant accéder à la socket.
- Fonction à utiliser uniquement pour le programme serveur (client utilise fonction ***connect()***)
- Retourne : 0 si succès, -1 si erreur

Communication en mode connecté

Connexion du client sur une application serveur

```
#include <sys/types.h>
#include <sys/socket.h>

int connect(int sockfd, struct sockaddr *serv_addr, int addrlen);
```



Socket du client

Structure avec adresse serveur et numéro port application

Taille structure serv_addr

- Client essaie de se connecter à une application serveur
- Retourne 0 si succès, -1 sinon

Communication en mode connecté

Ecoute et acceptation connexion de clients

```
#include <sys/types.h>
#include <sys/socket.h>
```

```
int listen(int sockfd, int backlog);
```

Socket d'écoute côté serveur

Nb max de connexion en attente de traitement (20 max système, en pratique entre 5 et 10)

- Serveur écoute et est en attente d'une demande de connexion d'un client
- Retourne 0 si succès, -1 sinon

```
newfd = accept(int sockfd, struct sockaddr *client_addr, socklen_t *addrlen);
```

Socket d'écoute côté serveur

Structure avec adresse et port du client

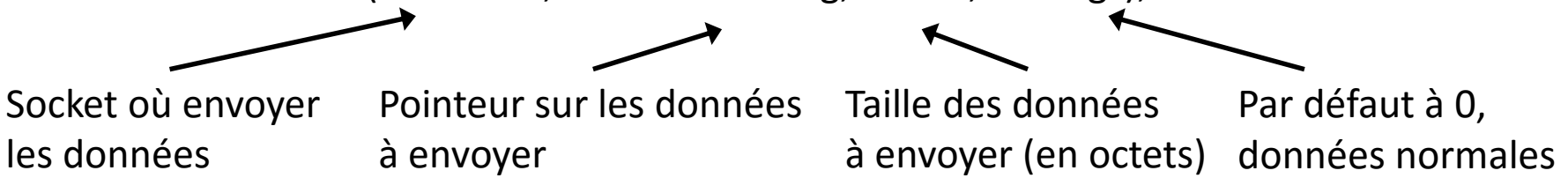
Taille structure client_addr

- Accepte une connexion client et retourne nouveau descripteur de socket de service (newfd) pour traiter requête du client
- Retourne -1 si echec

Communication en mode connecté

Envoie et réception de données sur une socket

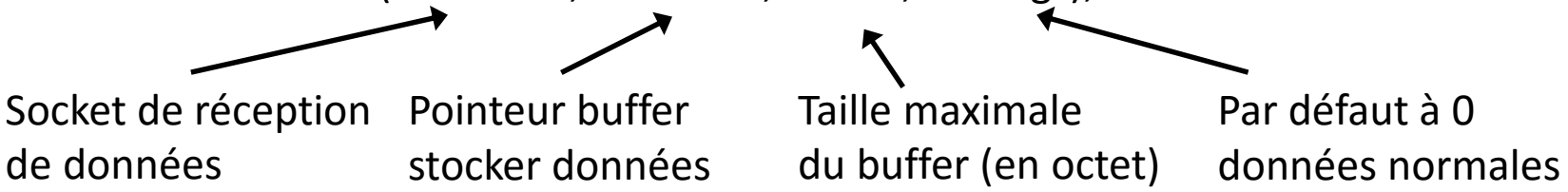
```
int send(int sockfd, const void *msg, int len, int flags);
```



- Envoie des données sur sockfd indiquées par pointeur msg
- Retourne nombre d'octets envoyés si succès (***peut être < len***), -1 sinon

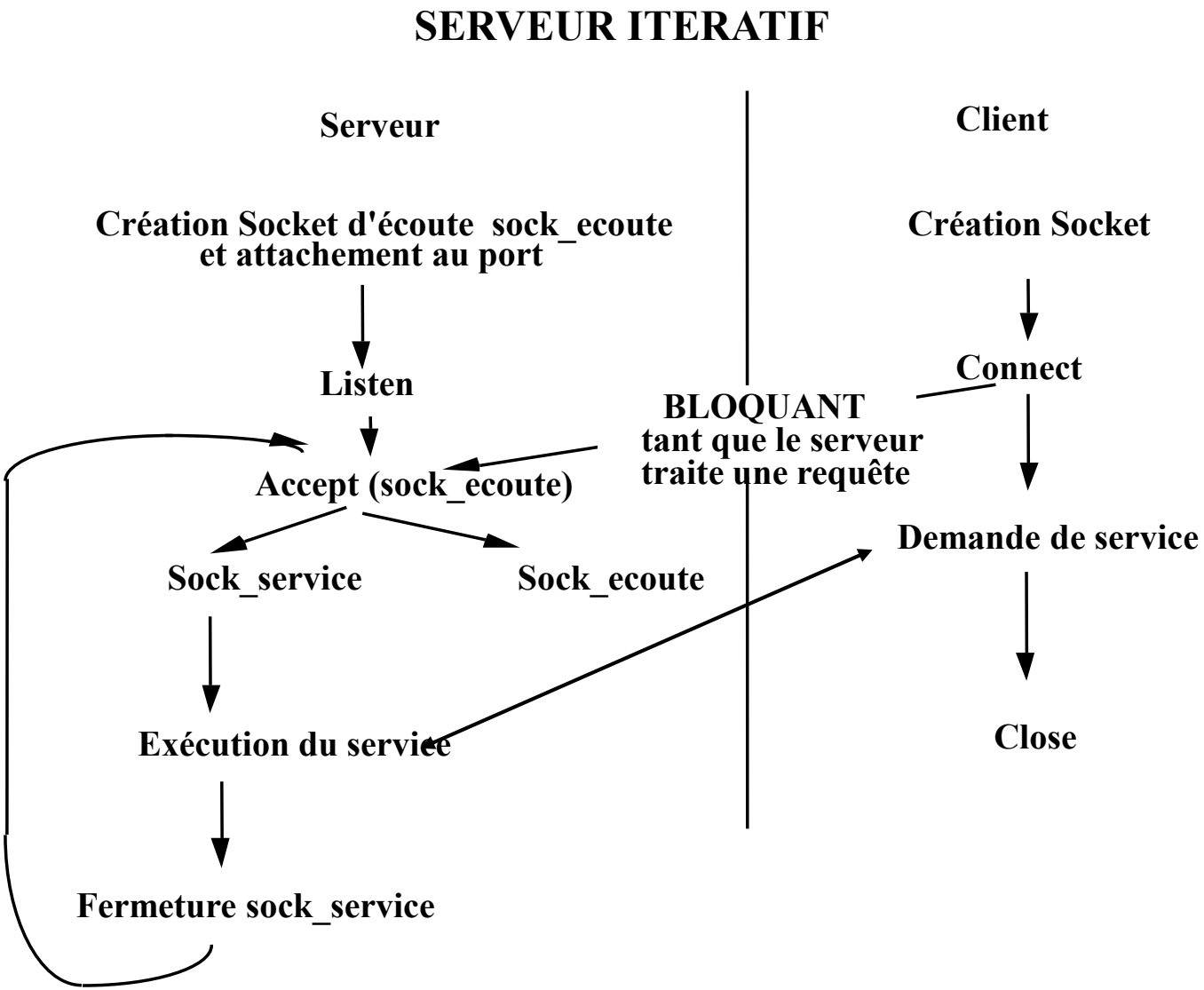
---

```
int recv(int sockfd, void *buf, int len, int flags);
```



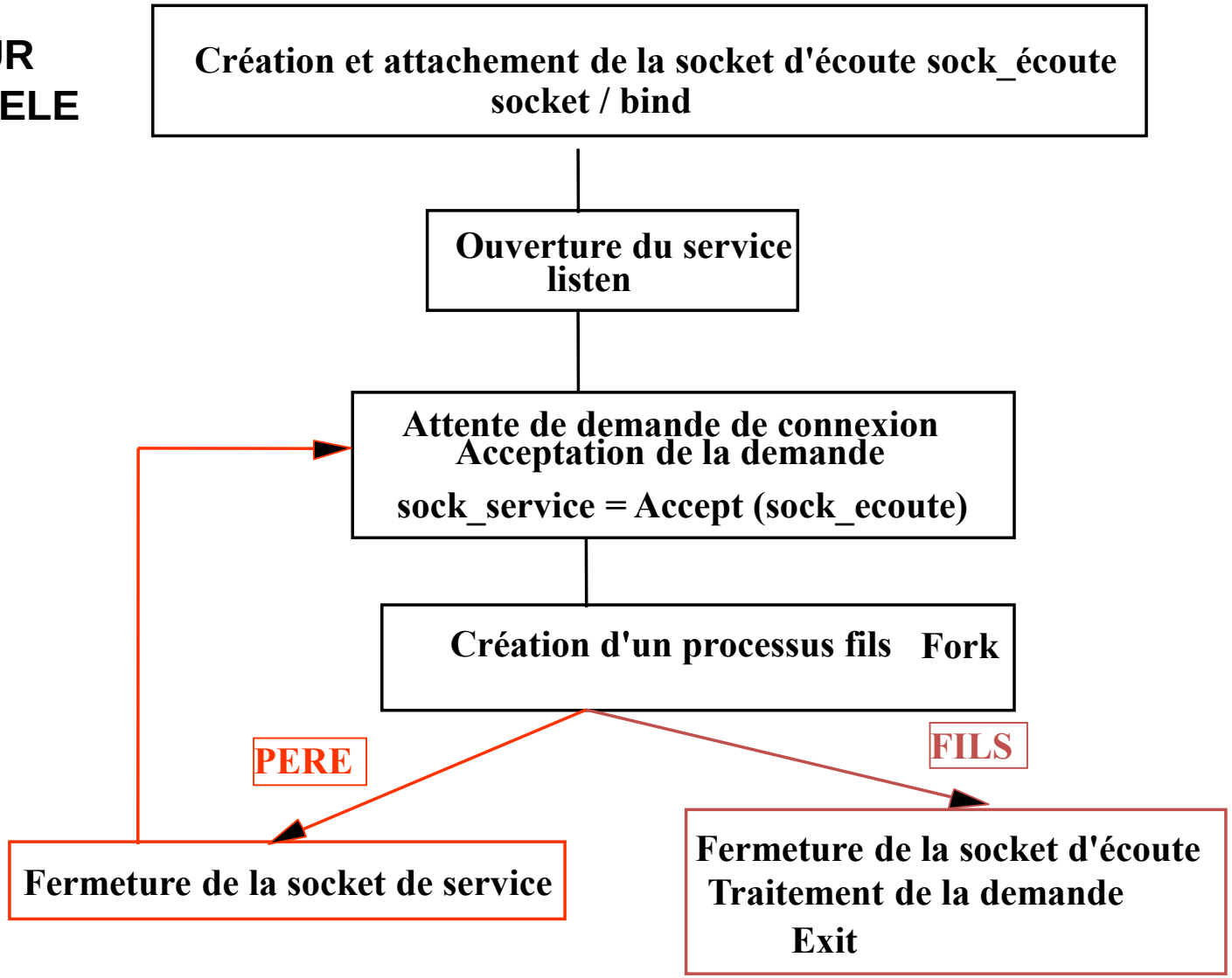
- Réceptionne des données d'une socket et les stocke dans un buffer (***appel bloquant***)
- Retourne nb octets lu si succès, 0 indique socket a été fermée, -1 sinon

Communication en mode connecté



Communication en mode connecté

SERVEUR PARALLELE



Communication en mode connecté

```
/******* SERVEUR TCP  **************/

#include      <stdio.h>
#include      <sys/types.h>
#include      <sys/socket.h>
#include      <netinet/in.h>
#include      <netdb.h>
#include      <netdb.h>

#define PORTS “2058”

main()
{
int sockfd, new_fd, rv, sin_size;
struct addrinfo hints, *servinfo, *p;
struct sockaddr their_adr;

memset(&hints, 0, sizeof(hints));
hints.ai_family = AF_UNSPEC;
hints.ai_socktype = SOCK_STREAM;
hints.ai_flags = AI_PASSIVE;              // use my IP
rv = getaddrinfo(NULL, PORTS, &hints, &servinfo);

if (rv != 0)
{ fprintf(stderr, "getaddrinfo: %s\n", gai_strerror(rv));
return 1;}
```

```
// Création  socket  et  attachement
for(p = servinfo; p != NULL; p = p->ai_next)
{
   if ((sockfd = socket(p->ai_family, p->ai_socktype,
   p->ai_protocol)) == -1) {
      perror("server: socket");
      continue;}
   if (bind(sockfd, p->ai_addr, p->ai_addrlen) == -1)
   {    close(sockfd);
        perror("server: bind");
        continue;}
   break;}
if (p == NULL) {
   fprintf(stderr, "server: failed to bind\n");
   return 2;}
freeaddrinfo(servinfo);    // Libère structure

listen(sockfd, 5);

while(TRUE)
{   sin_size = sizeof(their_addr);
   new_fd = accept(sockfd, &their_addr, &sin_size);
   if(!fork())
   {  close(sockfd);
      send(new_fd, ’’Hello!’’, 6, 0);
      close(new_fd);  exit(0);  } } }
```

Communication en mode connecté

```
/*********** CLIENT TCP  **************/

#include      <stdio.h>
#include      <sys/types.h>
#include      <sys/socket.h>
#include      <netinet/in.h>
#include      <netdb.h>
#include      <netdb.h>
#define SERVEUR “127.0.0.1”
#define PORTS “2058”

main()
{
int sockfd, new_fd, rv, sin_size;
struct addrinfo hints, *servinfo, *p;
struct sockaddr their_adr;
Char buf[100];

memset(&hints, 0, sizeof(hints));
hints.ai_family = AF_UNSPEC;
hints.ai_socktype = SOCK_STREAM;
rv = getaddrinfo(SERVEUR, PORTS, &hints, &servinfo);

if (rv != 0)
{ fprintf(stderr, "getaddrinfo: %s\n", gai_strerror(rv));
return 1;}
```

```
// Création  socket  et  attachement
for(p = servinfo; p != NULL; p = p->ai_next)
{
   if ((sockfd = socket(p->ai_family, p->ai_socktype,
   p->ai_protocol)) == -1) {
      perror(« client: socket");
      continue;}
   if ((connect(sockfd, p->ai_addr, p->ai_addrlen) == -1) {
      close(sockfd);
      perror("client: connect");
      continue;}
   break;}
if (p == NULL) {
   fprintf(stderr, "server: failed to bind\n");
   return 2;}
freeaddrinfo(servinfo);    // Libère structure

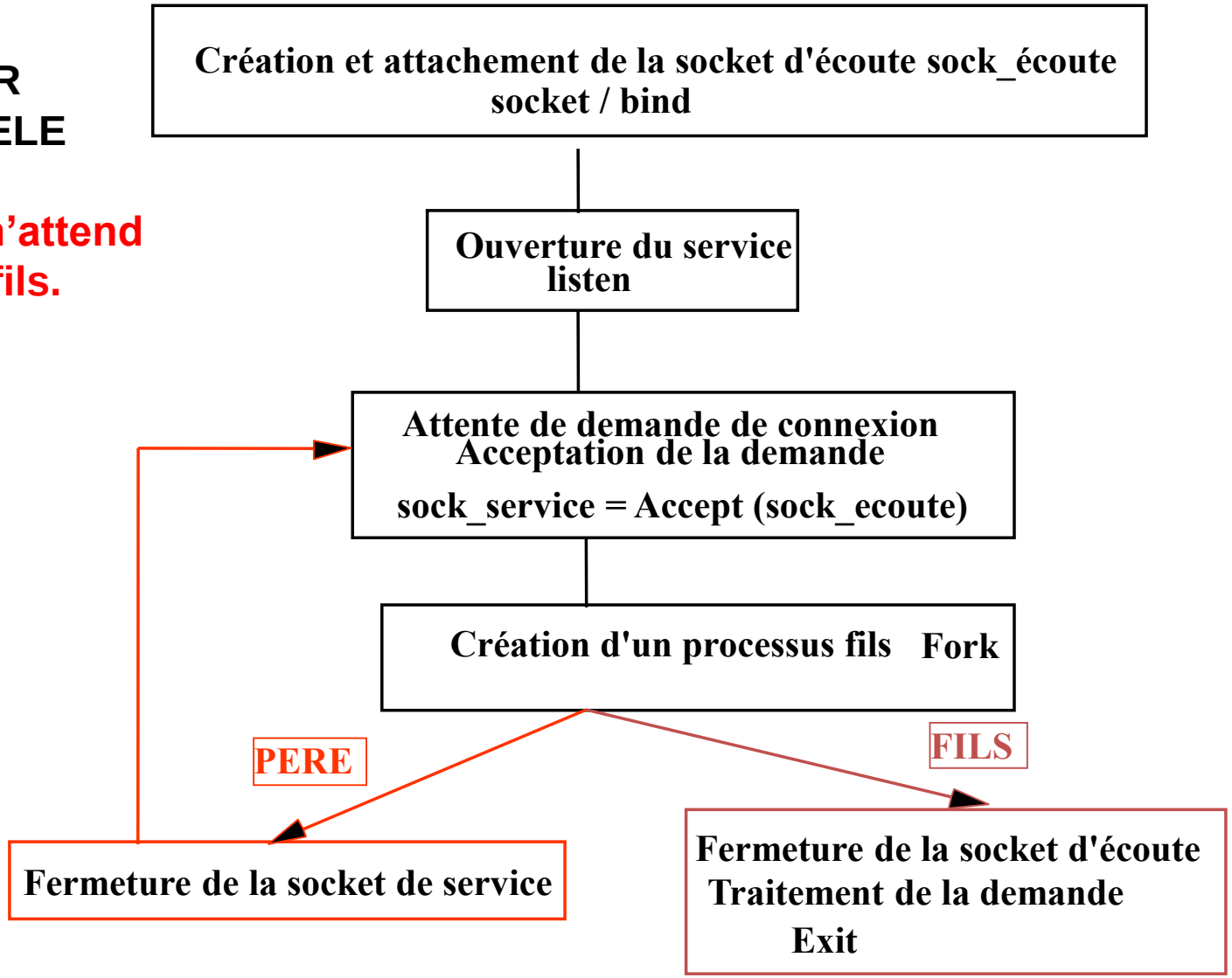
if ((numbytes = recv(sockfd, buf, 100-1, 0)) == -1)
{   perror("recv");
   exit(1);
}
printf(“’Message reçu : %s\n’’,buf);

close(sockfd);
return 0;
}
```

Communication en mode connecté

SERVEUR PARALLELE

Le père n'attend pas son fils.



Communication en mode connecté

```
/******* SERVEUR TCP  **************/
#include      <stdio.h>
#include      <sys/types.h>
#include      <sys/socket.h>
#include      <netinet/in.h>
#include      <netdb.h>
#include      <netdb.h>
#include      <signal.h>

#define PORTS “2058”

main()
{
int sockfd, new_fd, rv, sin_size;
struct addrinfo hints, *servinfo, *p;
struct sockaddr their_adr;

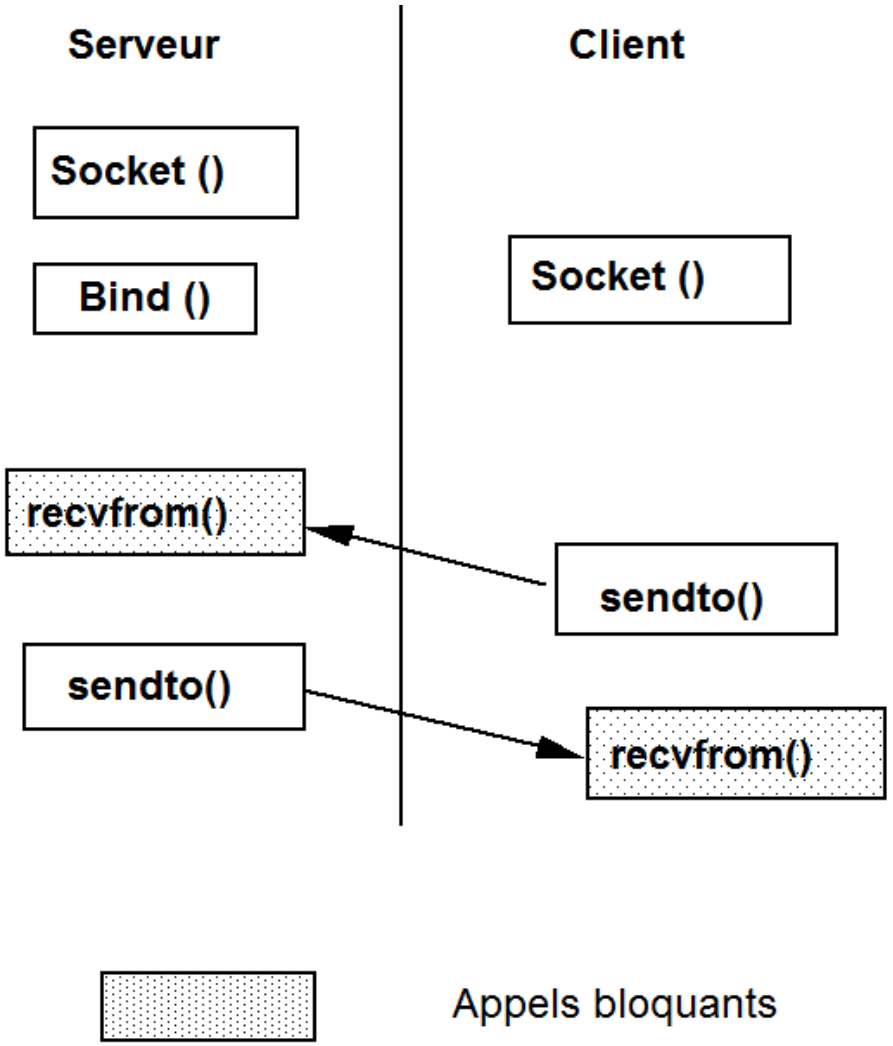
memset(&hints, 0, sizeof(hints));
hints.ai_family = AF_UNSPEC;
hints.ai_socktype = SOCK_STREAM;
hints.ai_flags = AI_PASSIVE;              // use my IP
rv = getaddrinfo(NULL, PORTS, &hints, &servinfo);

if (rv != 0)
{ fprintf(stderr, "getaddrinfo: %s\n", gai_strerror(rv));
return 1;}
```

```
// Création  socket  et  attachement
for(p = servinfo; p != NULL; p = p->ai_next)
{
   if ((sockfd = socket(p->ai_family, p->ai_socktype,
   p->ai_protocol)) == -1) {
      perror("server: socket");
      continue;}
   if (bind(sockfd, p->ai_addr, p->ai_addrlen) == -1)
   {   close(sockfd);
       perror("server: bind");
       continue;}
   break;}
if (p == NULL) {
   fprintf(stderr, "server: failed to bind\n");
   return 2;}
freeaddrinfo(servinfo);   // Libère structure

listen(sockfd, 5);
signal(SIGCHLD, SIG_IGN);
while(TRUE)
{   sin_size = sizeof(their_addr);
   new_fd = accept(sockfd, &their_addr, &sin_size);
   if(!fork())
   {  close(sockfd);
      send(new_fd, ’’Hello!’’, 6, 0);
      close(new_fd);  exit(0);  } } }
```

Communication en mode datagramme



Sock = socket (AF_INET, SOCK_DGRAM, IPPROTO_UDP)

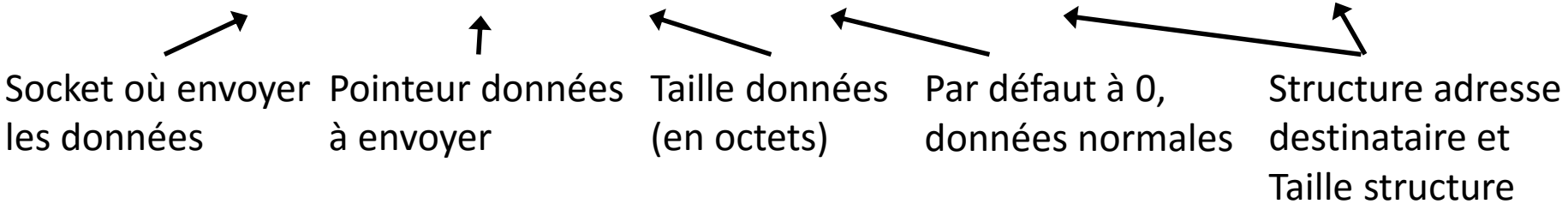
Pas d'établissement de connexion

→ Chaque message échangé contient l'adresse du destinataire

Communication en mode non-connecté

Envoie et réception de données sur une socket

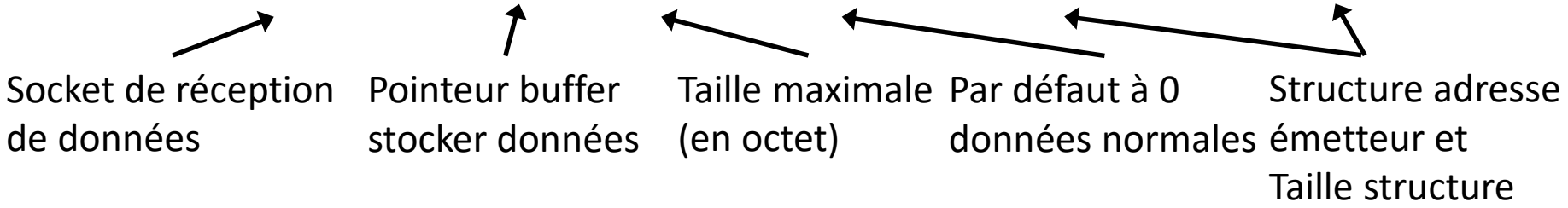
int sendto(int sockfd, void *msg, int len, int flags, struct sockaddr *to, socklen_t tolen);



- Envoie des données sur sockfd indiquées par pointeur msg
- Retourne nombre d'octets envoyés si succès (***peut être < len***), -1 sinon

---

int recvfrom(int sockfd, void *buf, int len, int flags, struct sockaddr *from, int *fromlen);



- Réceptionne des données d'une socket et les stocke dans un buffer (***appel bloquant***)
- Retourne nb octets lu si succès, 0 indique socket a été fermée, -1 sinon

Communication en mode non-connecté

```
/******* SERVEUR UDP  **************/
#include      <stdio.h>
#include      <sys/types.h>
#include      <sys/socket.h>
#include      <netinet/in.h>
#include      <netdb.h>
#include      <netdb.h>

#define PORTS “2058”

main()
{
int sockfd, new_fd, rv, addr_len;
struct addrinfo hints, *servinfo, *p;
struct sockaddr their_adr;
char buf[100];

memset(&hints, 0, sizeof(hints));
hints.ai_family = AF_UNSPEC;
hints.ai_socktype = SOCK_DGRAM;
hints.ai_flags = AI_PASSIVE;              // use my IP
rv = getaddrinfo(NULL, PORTS, &hints, &servinfo);

if (rv != 0)
{ fprintf(stderr, "getaddrinfo: %s\n", gai_strerror(rv));
return 1;}
```

```
// Création  socket  et  attachement
for(p = servinfo; p != NULL; p = p->ai_next)
{
   if ((sockfd = socket(p->ai_family, p->ai_socktype,
   p->ai_protocol)) == -1) {
      perror("server: socket");
      continue;}
   if (bind(sockfd, p->ai_addr, p->ai_addrlen) == -1)
   {    close(sockfd);
        perror("server: bind");
        continue;}
   break;}
if (p == NULL) {
   fprintf(stderr, "server: failed to bind\n");
   return 2;}
freeaddrinfo(servinfo);    // Libère structure

while(TRUE)
{
   if ((numbytes = recvfrom(sockfd, buf, 100-1, 0,
                                &their_addr, &addr_len)) == -1)
   {   perror("recv");
      exit(1);}
   printf("Chaine reçue %s\n", buf);
}
close(sockfd);  } }
```

Communication en mode non-connecté

```
/*********** CLIENT UDP  **************/
#include      <stdio.h>
#include      <sys/types.h>
#include      <sys/socket.h>
#include      <netinet/in.h>
#include      <netdb.h>
#include      <netdb.h>
#define SERVEUR “127.0.0.1”
#define PORTS “2058”

main()
{
int sockfd, new_fd, rv, sin_size;
struct addrinfo hints, *servinfo, *p;
struct sockaddr their_adr;
Char buf[100];

memset(&hints, 0, sizeof(hints));
hints.ai_family = AF_UNSPEC;
hints.ai_socktype = SOCK_DGRAM;
rv = getaddrinfo(SERVEUR, PORTS, &hints, &servinfo);

if (rv != 0)
{ fprintf(stderr, "getaddrinfo: %s\n", gai_strerror(rv));
return 1;}
```

```
// Création  socket  et  attachement
for(p = servinfo; p != NULL; p = p->ai_next)
{
   if ((sockfd = socket(p->ai_family, p->ai_socktype,
   p->ai_protocol)) == -1) {
      perror(« client: socket");
      continue;}
   break;}
if (p == NULL) {
   fprintf(stderr, "server: failed to bind\n");
   return 2;}
freeaddrinfo(servinfo);    // Libère structure

if ((numbytes = sendto(sockfd, ‘’HELLO!’’, 6, 0, p->ai_addr,
p->ai_addrlen)) == -1)
{   perror("sendto");
   exit(1);
}

close(sockfd);
return 0;
}
```

Sockets avancés

Fonctions bloquantes

- Certaines fonctions de l'API Socket sont bloquantes
  - Attente de l'arrivée d'un évènement
  - Exemple de fonctions
    - *connect, accept, recv/recvfrom*
- Besoin parfois de socket non bloquant
  - Attente d'évènements sur plusieurs sockets
- Possibilité de rendre socket non bloquant
  - Fonction *fcntl()*
  - Pb : utilisation attente active pour recevoir données
    - Utilisation inutile du CPU

Utilisation de plusieurs Sockets

- Fonction donnant l'état de plusieurs sockets

```
int select(int numfds, fd_set *readfds, fd_set *writefds, fd_set *exceptfds, struct timeval *timeout);
```

Numéro plus grand Socket descriptor +1

Ensembles de sockets en attente de :
- Lecture
- Ecriture
- Exceptions

Timer attente, Désactivé si NULL

- Fonction attendant évènement sur l'un des sockets des ensembles (ou attente période indiquée par ***timeout***)
- Retourne nb sockets avec évènements, 0 si timeout, -1 si erreur

```
struct timeval {
        int tv_sec; // seconds
        int tv_usec; // microseconds
};
```

Utilisation des ensembles de Sockets

- Plusieurs macro utiles
  - Ajout d'un descripteur de socket
    - **FD_SET(int fd, fd_set *set);**
  - Suppression d'un descripteur de socket
    - **FD_CLR(int fd, fd_set *set);**
  - Test si descripteur socket est dans l'ensemble
    - **FD_ISSET(int fd, fd_set *set);**
  - Supprime tous les éléments de l'ensemble
    - **FD_ZERO(fd_set *set);**

Exemple utilisation Select()

```
/******* SERVEUR TCP  **************/
#include      <stdio.h>
#include      <sys/types.h>
#include      <sys/socket.h>
#include      <netinet/in.h>
#include      <netdb.h>

#define PORTS “2058”

main()
{
int sockfd, new_fd, rv,;
struct addrinfo hints, *servinfo, *p;
struct sockaddr their_adr;
socklen_t addrlen;
char buf[256];  // buffer données client

fd_set master;  // master file descriptor list
fd_set read_fds;  // temp file descriptor list for select()
int fdmax;  // maximum file descriptor number

FD_ZERO(&master);  // clear the master and temp sets
FD_ZERO(&read_fds);

// Création  socket  et  attachement
                ……..
listen(sockfd, 5);

FD_SET(sockfd, &master);   // Ajout sockfd à ensemble
fdmax = sockfd;   // Garde valeur max socket

while(TRUE)
{  read_fds = master;     // ensemble socket attente lecture
   if (select(fdmax+1, &read_fds, NULL, NULL, NULL) == -1)
   {  perror("select"); exit(4);}

   for(i = 0; i <= fdmax; i++)
   {
      if (FD_ISSET(i, &read_fds))
      {   if (i == sockfd)
          {       addrlen = sizeof(their_adr);
                  new_fd = accept(sockfd,
                                  &their_adr, &addrlen);
                  if (new_fd == -1)
                  {  perror("accept");}
                  else
                  {  // Ajout new_fd  à ensemble
                     FD_SET(new_fd, &master);
                     if (new_fd > fdmax)
                     {  fdmax = new_fd; }
                     printf(’’Nouvelle connexion au serveur.\n’’);
                  }
   }
}
```

Exemple utilisation Select()

```
else
{   // gestion données client i
   if ((nbytes = recv(i, buf, sizeof buf, 0)) <= 0)
   {   // erreur ou connexion fermée par client
      if (nbytes == 0)
      { printf(« Connexion %d fermée.\n", i);}
      else
      {  perror("recv"); }
      close(i);
      FD_CLR(i, &master); // Supprime ensemble
    }
    else
    {  // Données reçu du client
       for(j = 0; j <= fdmax; j++)
       {  // Envoie données à tous les autres clients j
          if (FD_ISSET(j, &master))
          {
              // Sauf serveur et client source données
              if (j != sockfd && j != i)
              {
                  send(j, buf, nbytes, 0);
              }
          }
       }
     }
          } // Fin bloc ELSE client
       } // Fin bloc IF  FD_ISSET
    } // Fin bloucle FOR sur i
} // Fin boucle WHILE

 return 0;
}
```

Echange de données

- Envoie de données en mode caractères
  - Utilisation fonction *sprintf / snprintf*
- Définition d'un format de message
  - Type du message
  - Taille total du message
    - Utiliser fonction *htons()*
  - Nom expéditeur (nb caractères max fixé)
    - Complété par caractère '\0' si inférieur à max
  - Données (nb caractères max fixé)

Envoie/Réception données

- Envoie
  - Faire attention lors de l'envoi
    - **Tester nombre caractères réellement envoyés !**
  - Possible de faire appel plusieurs fois à *send()*
    - Si message à envoyer dépasse taille max paquet
- Réception
  - Utiliser taille message max avec fonction *recv()*
    - Attention indication taille max tampon et pas taille message à recevoir